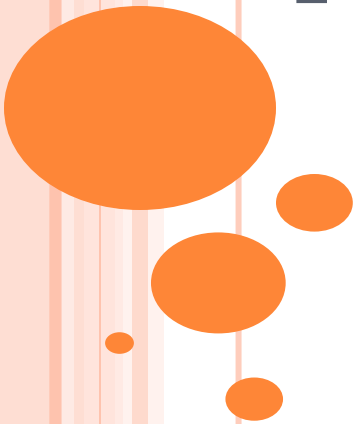
Fundamentos de Spring

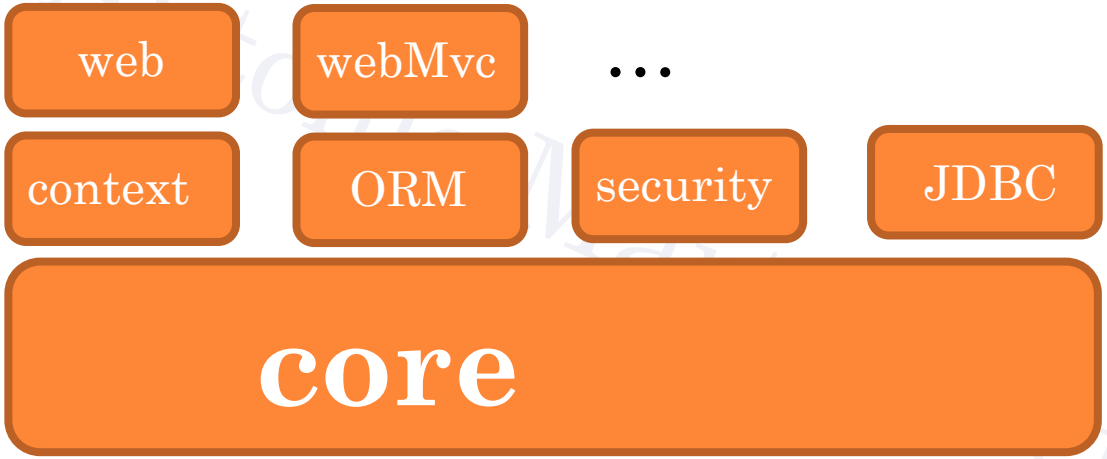


¿Qué es Spring?

- Framework para la construcción de aplicaciones empresariales
- Puede ser utilizado en todas las capas de la aplicación
- Está organizado en módulos, solo se incorporarán aquellas dependencias de los módulos que se necesiten
- Basado en la filosofía de menos programación, más configuración

Antonio Martín Sierra

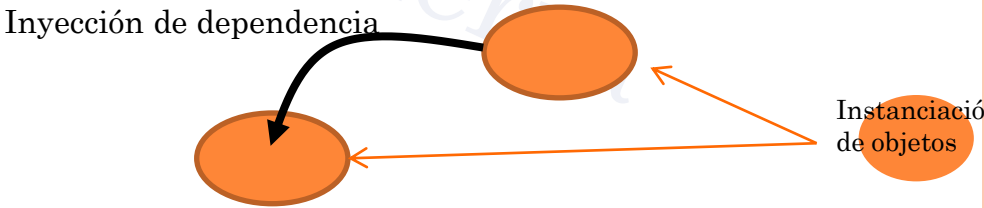
Módulos Spring



El módulo Spring Core

- **Es incluido en todas las aplicaciones**
- **Proporciona la funcionalidad básica de Spring:**

Instanciación de objetos	• Spring se encarga de instancias clases anotadas con @Controller, @Repository, @Service, etc.
Inyección de dependencia	• Las instancias creadas por Spring son inyectadas automáticamente en variables de otras instancias mediante @Autowired

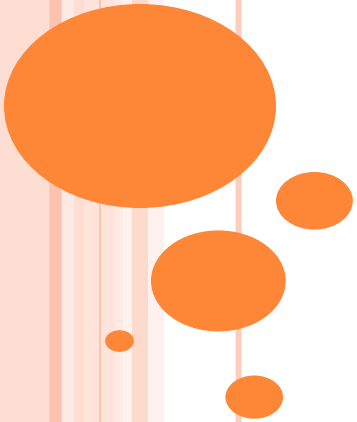


Dependencias

- **Incorporación vía Maven**
  - **core** — Todas las aplicaciones
  - **context** — Todas las aplicaciones
  - **web** — Aplicaciones Web
  - **webmvc** — Aplicaciones Web
  - **jdbc** — Acceso a datos con JDBC
  - **orm** — Acceso a datos mediante persistencia
  - **security** — Securización aplicaciones

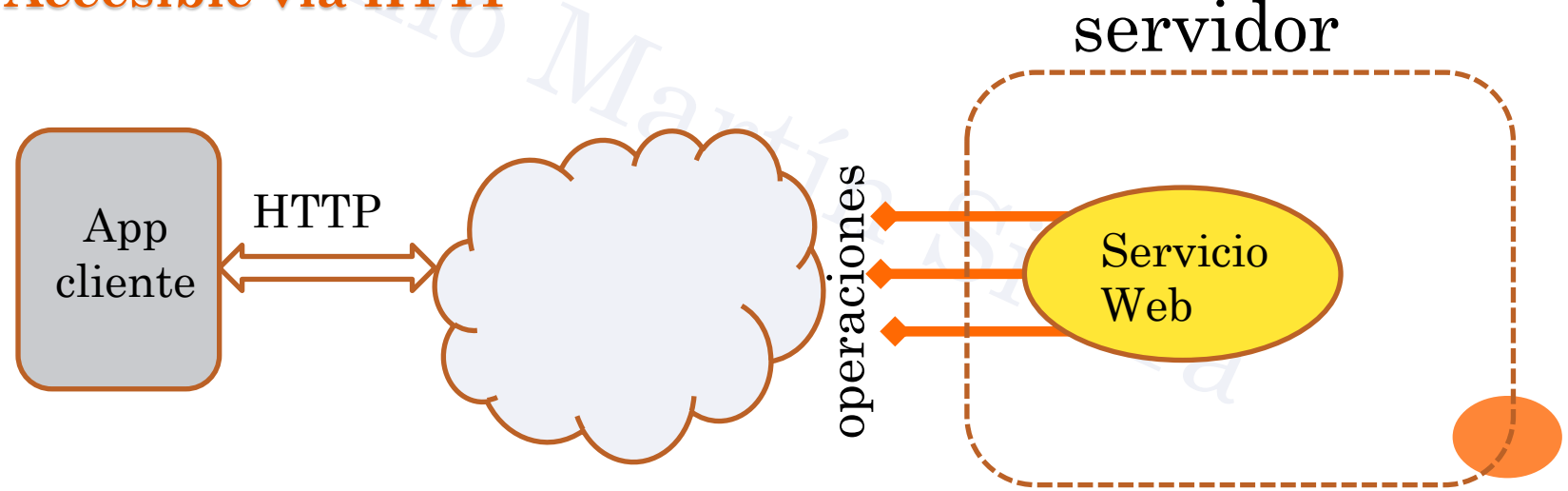
Antonio Martín Sierra

Servicios Web



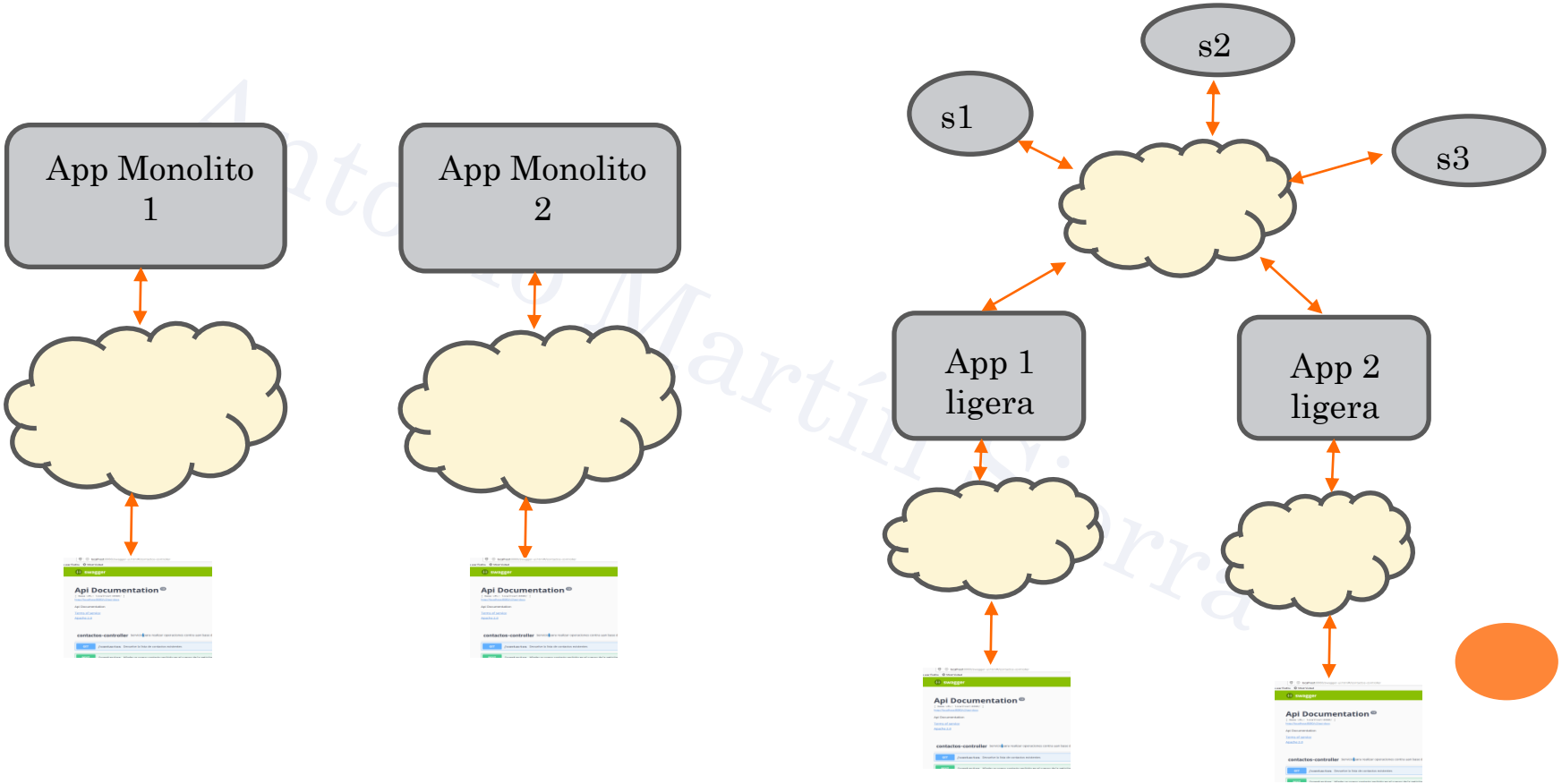
¿Qué es un servicio Web?

- Componente que expone operaciones a otros programas a través de la Web
- Accesible vía HTTP



Antonio Martín Sierra

Servicios vs aplicaciones

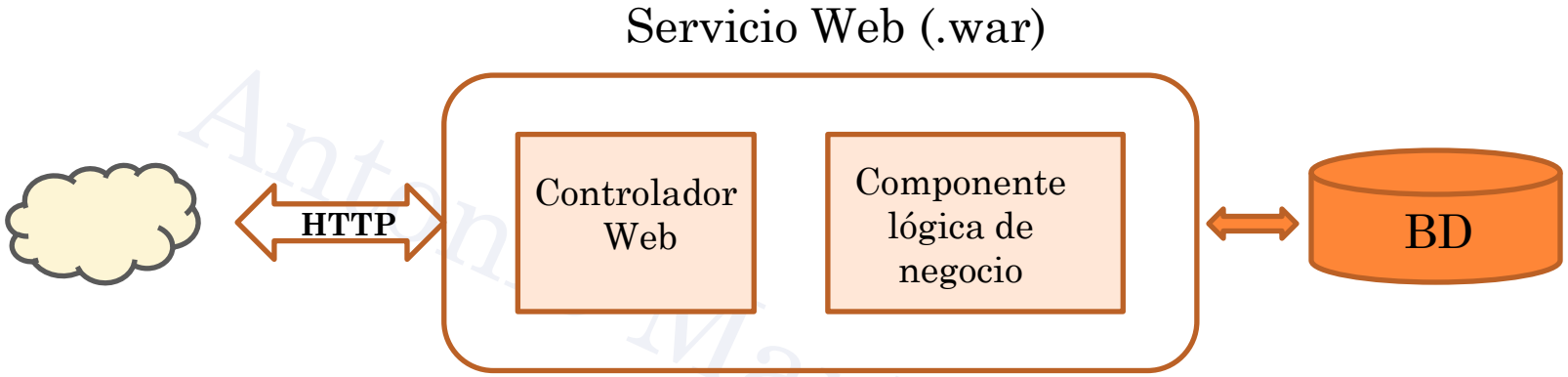


Ventajas servicios Web

- Reutilización de código.
- Independencia de la plataforma
- Escalabilidad
- Mejora de rendimiento

Antonio Martín Sierra

Estructura de un servicio Web



Antonio Martín Sierra

- **Lógica de negocio. Implementación de las operaciones**
- **Controlador Web. Interacción con el exterior: Mapeo de datos nativos a formato de intercambio (XML, JSON,..), gestión de conexiones, etc.**

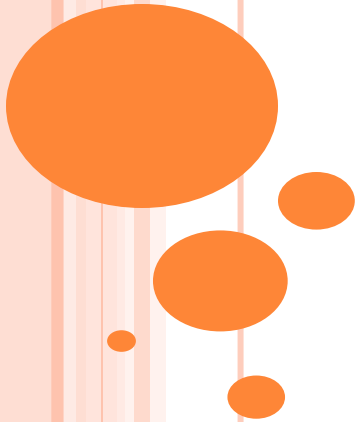
Aproximaciones

- ***Servicios Web XML.*** **Se basan en el intercambio de documentos XML entre el servicio y su aplicación cliente.**

- ***Servicios Rest.*** **Los servicios web exponen una serie de recursos a los que se accede simplemente con una petición HTTP, sin necesidad de enviar ningún documento XML adicional. Los servicios REST pueden aceptar datos y devolver resultados en múltiples formatos**

Antonio Martín Sierra

Servicios Rest



Fundamentos REST

- Los servicios REST (Representational State Transfer), exponen a través de Internet una serie de recursos a los que se puede acceder con peticiones HTTP sencillas (tipo GET, PUT, POST, DELETE, etc.)
- Los recursos consisten en datos que se ofrecen (pedidos, pedido de un determinado identificador) u operaciones sobre esos datos (modificar un pedido, añadir un pedido,..)
- Cada recurso se identifica por: URL, método HTTP, variables, tipo de respuesta, tipo consumido

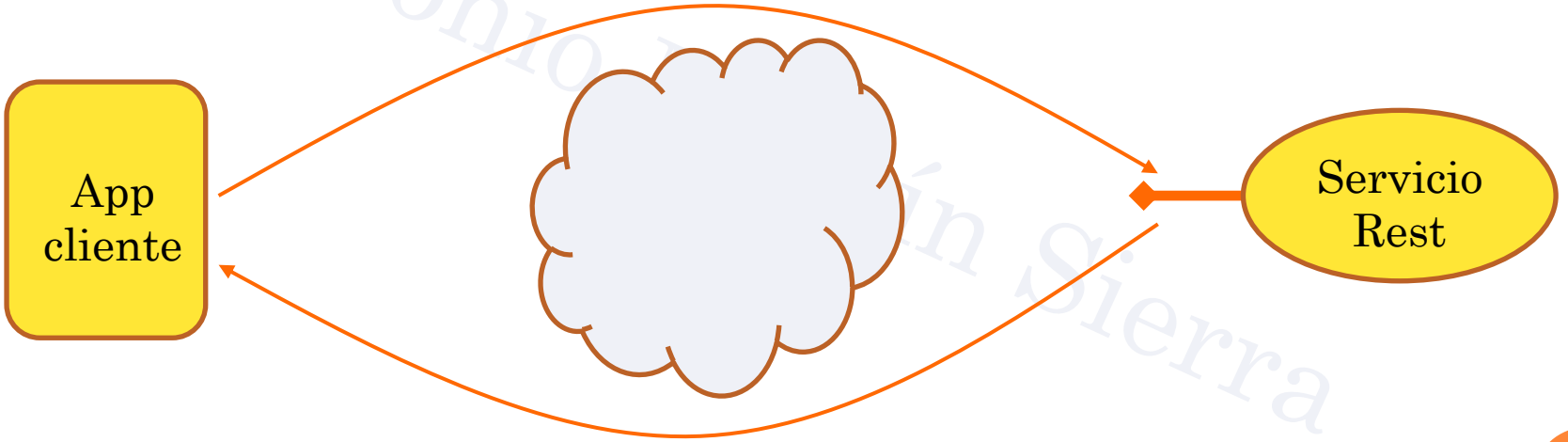
Antonio Martín Sierra

Formato de intercambio de datos

URL:http://servidor:puerto/app/recurso/var1/var2?par1=valor1

Petición HTTP (GET,POST,PUT,DELETE)

	Params, XML, JSON,..



Respuesta HTTP

	Text, XML, JSON,..

Antonio J. Sierra

Ventajas de REST

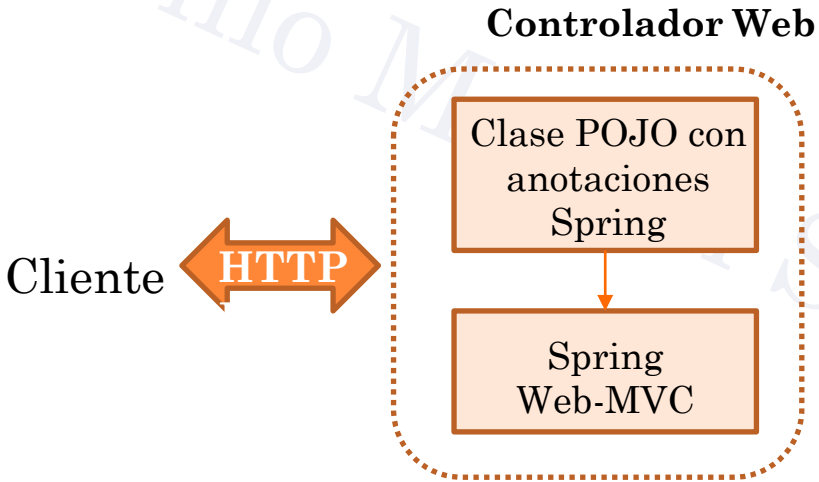
Entre las ventajas que nos proporciona la arquitectura de servicios REST frente a los servicios XML podemos destacar:

- **Acceso más simple**. Para acceder a un servicio REST **no necesitamos incluir ningún documento XML en el cuerpo de la petición**, basta con enviar una simple petición HTTP, que hará que se ejecute una operación dentro del servicio.
- **Flexibilidad**. Los servicios REST nos ofrecen una gran flexibilidad a la hora de intercambiar datos con la aplicación cliente, puesto que se admiten múltiples formatos de. Los servicios web REST no solo admiten datos en XML, sino también en **JSON**, binario, texto plano, etc. Además, la forma de estos datos puede ser cualquiera, no tiene que ajustarse a una determinada estructura. Y lo mismo sucede con los datos de la respuesta.
- **Eficiencia**. La construcción y manipulación de documentos XML resulta bastante tediosa y consume muchos recursos de la máquina. Al poder emplear otros formatos de datos mucho más eficientes se mejora el rendimiento de las aplicaciones.

Ricardo Moreno Sierra

Servicios REST con Spring

- El módulo Web-MVC proporciona el soporte necesario para la creación de servicios REST con Spring
- Incluye anotaciones específicas de Spring



Antonio M. Sierra

Estructura controlador REST

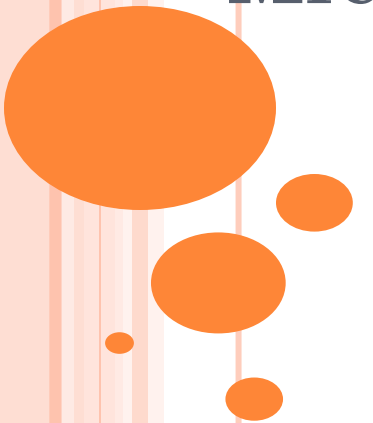
- **Clase POJO con anotaciones spring MVC**

```
@RestController
public class ClaseServicio{
        @GetMapping(..)
        public String metodo(..){..}

        @PostMapping(..)
        public List<> metodo2(..){..}
        :
}
```

Tipo del dato enviado como respuesta en el body

Microservicios y servicios



¿Qué es un microservicio?

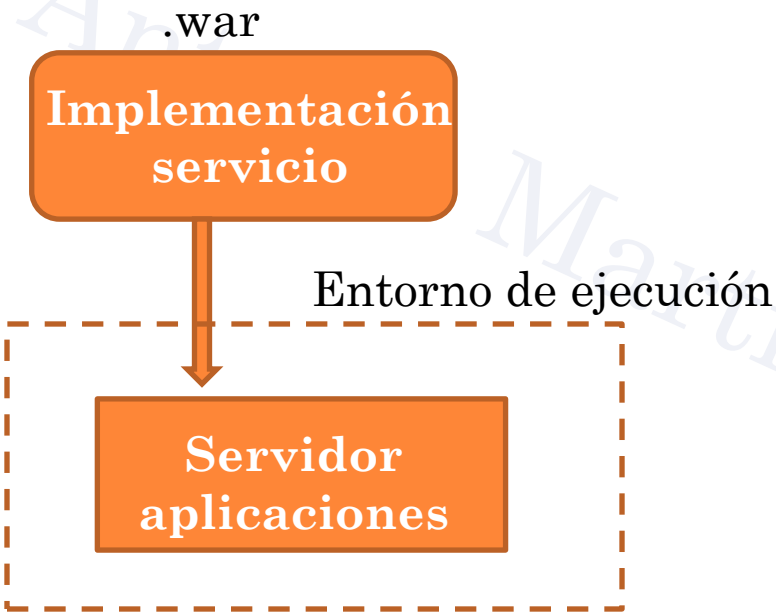
Microservicio=Servicio REST+servidor app

- **Un microservicio es una unidad de software que implementa una funcionalidad concreta y se ejecuta de forma autónoma e independiente**
- **Incluye todo lo necesario para su ejecución, sin depender de ningún software adicional**
- **Despliegue automático e independiente**
- **Ideal entornos cloud**

Antonio Martín Sierra

Microservicio vs servicio

App servicio Rest estándar



App microservicio



Antonio Martín Sierra

Microservicios con spring boot

- Spring Boot permite crear aplicaciones de tipo microservicio a elegir la opción aplicación .jar
- El starter web incluye dependencias de apache tomcat, que se incluye en el desplegable final
- La clase lanzadora se encarga de iniciar tomcat y desplegar en él el microservicio

Antonio Martín Sierra

Dependencias Maven

- La inclusión de dependencias en una aplicación Spring boot se simplifica mediante los starters

- Un starter incluye un conjunto de dependencias básicas maven para desarrollar un tipo de aplicación

- Ejemplo:

Incluye Spring core, context, web, web-mvc y jackson

```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Antonio Martín Sierra

Configuración de aplicaciones

- Se eliminan los archivos de configuración .xml y las clases de configuración se simplifican enormemente
- Se asumen una serie de configuraciones por defecto aplicadas por los starters
- Para indicar parámetros de configuración específicos:
  - application.properties
  - application.yml

Antonio Martín Sierra

La clase main

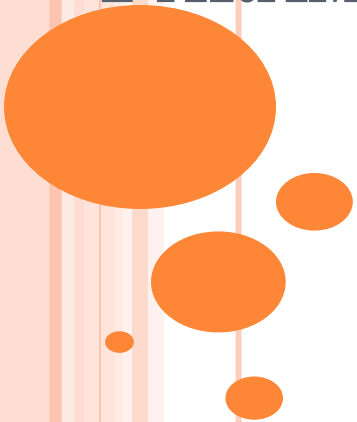
- Es la clase que inicia spring boot y pone en marcha todo el proceso de aplicación de configuraciones.

```
@SpringBootApplication
public class Application {
        public static void main(String[] args) {
                SpringApplication.run(Application.class, args);
        }
}
```

- Se utiliza también como clase de configuración.

- @SpringBootApplication equivale a tres anotaciones (@Configuration, @EnableAutoConfiguration y @ComponentScan).

Parámetros, variables y objetos



Variables en URL

- **Datos que se envían como parte de la URL, incluyéndolos a continuación de la dirección del servicio**



Antonio Martín S.

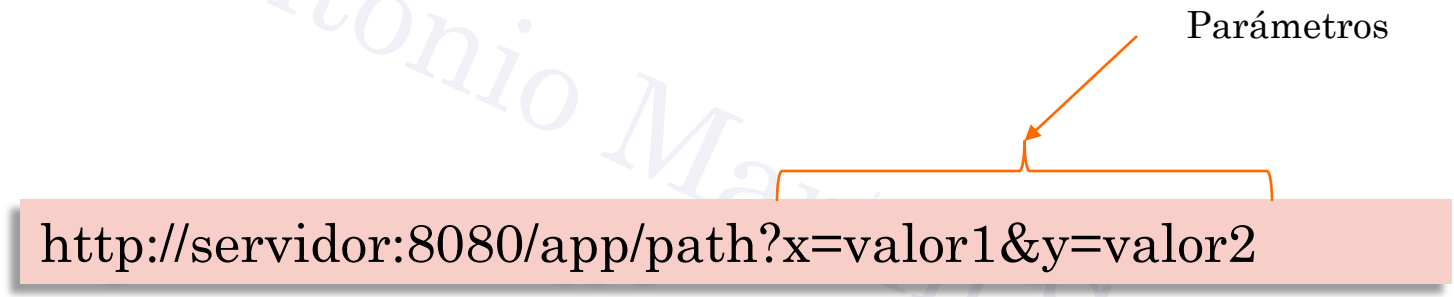
Recogida de variables

- **Los variables de la URL se deben mapear a los parámetros del método que procesa la petición mediante @PathVariable**

```
@RestController
public class TestService{
    @GetMapping(value="saludo/{x}/{y}", produces=MediaType.TEXT_PLAIN_VALUE)
    public String saludar(@PathVariable ("x")  String a, @PathVariable("y")  int b){
                :
    }
}
```

Parámetros en QueryString

- **Los parámetros se envían en parejas nombre=valor, separados de la dirección por una ?**



Antonio Martín Sierra

Recogida de parámetros

- **Los parámetros querystring se mapean a parámetros del método mediante @RequestParam**

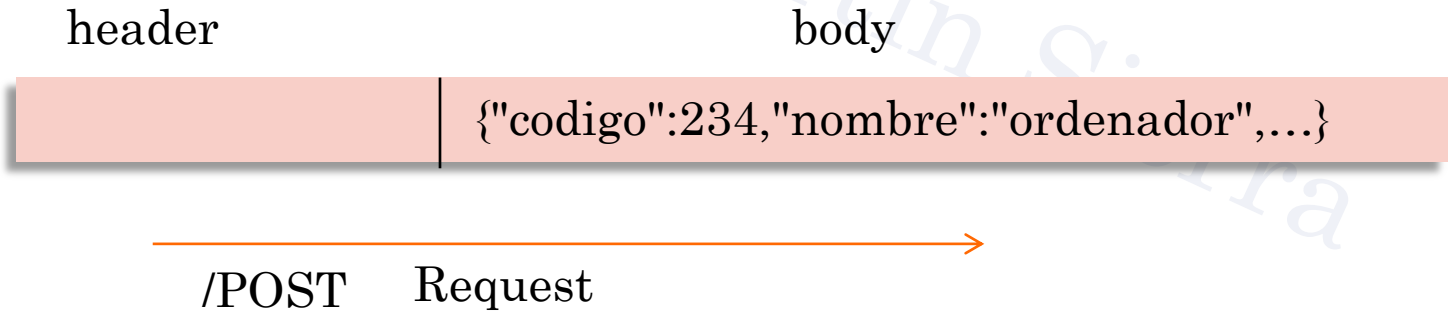
```
@RestController
public class TestService{
    @GetMapping(value="saludo", produces(MediaType.TEXT_PLAIN_VALUE)
    public String saludar(@RequestParam("x")  String a,@ RequestParam("y")  int b){
                :
    }
}
```

- **Si los parámetros son enviados por el cliente como form-urelencoded será recogidos de la misma manera**

Objetos JSON en body

- **En peticiones POST y PUT es posible enviar objetos en el cuerpo de la petición**
- **Normalmente, estos objetos se envían en formato JSON**

Antonio Martín Sierra



Recogida de objetos

- Los objetos del body se mapean directamente a objetos Java a través de la anotación @RequestBody

```
@RestController
public class TestService{
   @PostMapping(value="data", consumes=MediaType.APPLICATION_JSON_VALUE)
   public void process(@RequestBody  Producto producto){
            :
   }
}
```

- La clase Producto es un JavaBean con constructor sin parámetros

Respuestas completas

Tipo ResponseEntity<T>

- **Permite definir uns respuesta completa, especificando el cuerpo, encabezados y estado de la misma**

- **Para crear un objeto ResponseEntity, utilizaremos algunos de los siguientes constructores.**

ResponseEntity(HttpStatus status)
ResponseEntity(T body, HttpStatus status)
ResponseEntity(T body, MultiValueMap<String,String> headers, HttpStatus status)

cuerpo de la respuesta — mapa de encabezados — estado de la respuesta

- **También se puede emplear algunos métodos estáticos:**

•ok() -> Devuelve un builder configurado con código de estado 200
•ok(T body)-> Devuelve ResponseEntity con cuerpo T y código de estado 200
•status(HttpStatus status) -> Devuelve un builder con código de estado especificado

Antonio Martín Sierra

Tipo en el controlador

- **El tipo de devolución del método controlador será ResponseEntity:**

```
@GetMapping(value="contactos/{id}",produces=MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<Contacto> recuperarContactos(@PathVariable("id") int id) {
                    :
}
```

- **En caso de cuerpo vacío, se especificará tipo Void en ResponseEntity. El código de estado será utilizado para informar de alguna situación anómala:**

```
@PostMapping(value="contactos",consumes=MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<Void> altaContacto(@RequestBody contacto) {
        if(!contactosService.alta(contacto)){
            return ResponseEntity.status(HttpStatus.CONFLICT).build();
        }
        return ResponseEntity.ok().build();
}
```

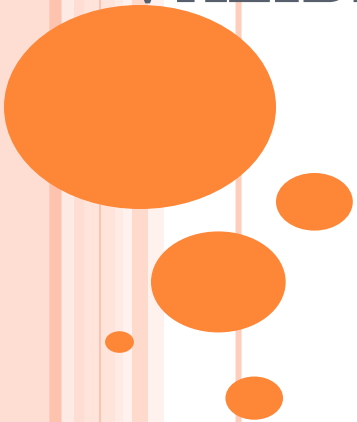
Respuestas ante excepciones

- Se capturan en una clase independiente, anotada con @RestControllerAdvice
- Cada tipo de excepción es gestionada por un método @ExceptionHandler, que devuelve ResponseEntity

Métodos para gestión de excepciones

```
@RestControllerAdvice
public class GestionExcepciones {
        @ExceptionHandler(TipoExcepcion1.class)
        public ResponseEntity<TipoRespuesta> manejaError1(TipoExcepcion1 e) {
                ...
        }
        @ExceptionHandler(TipoExcepcion2.class)
        public ResponseEntity<TipoRespuesta> manejaError2(TipoExcepcion2 e) {
                ...
        }
}
```

Validación de datos de entrada



JavaBean Validation

- Permite validar los datos enviados en el body de una petición antes de que sean procesados por el controlador
- Se requiere la dependencia:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

- Si tenemos el starter spring-boot-starter-test, no es necesario incluirlo, ya que se requiere transitivamente
- Permite validar parámetros, variables de url y campos de un bean

Uso de Validation en beans

- Se colocan las anotaciones en los campos del JavaBean:

```
public class Item {
    @NotBlank(message="La url no puede estar vacía")
    private String url;
    private String tematica;
    @Size(max=200, min=6, message="La descripción no  puede tener menos de 6 ni más de 200 ")
    private String descripcion;
    :
}
```

- Para aplicar las validaciones, se utiliza la anotación @Valid delante del objeto en el controlador:

```
@PostMapping(value="items",consumes=MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<Void> altaItem(@Valid @RequestBody Item item){
```

- Si se incumple alguna de las reglas de validación, el controlador devolverá directamente un error 400 - Bad Request

Uso en parametros

- Se colocan las anotaciones delante de los parámetros y variables URL del controlador

```
@GetMapping(value="alumnos/edades",produces=MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<Alumno> rangoEdad(@Positive @RequestParam int edad1,
                    @Min(10) @RequestParam int edad2){
      :
}
@GetMapping(value="alumnos/{codigo}",produces=MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<Alumno> buscar(@Positive @RequestParam int codigo){
      :
}
```

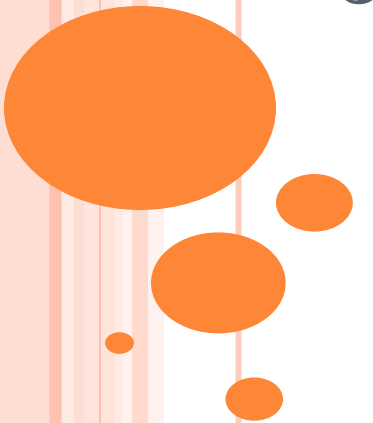
- De la misma forma que con el bean si se incumplen las validaciones se devuelve error 400

Anotaciones de Validation

- Entre las principales anotaciones tenemos:
  - @NotNull. Verifica que el campo no es null
  - @NotBlank. Que no sea cadena vacía
  - @Size. Que tenga un tamaño entre min y max caracteres
  - @Min y @Max. Rango numérico entre dos valores
  - @Email. Email válido
  - @Positive. Mayor que 0

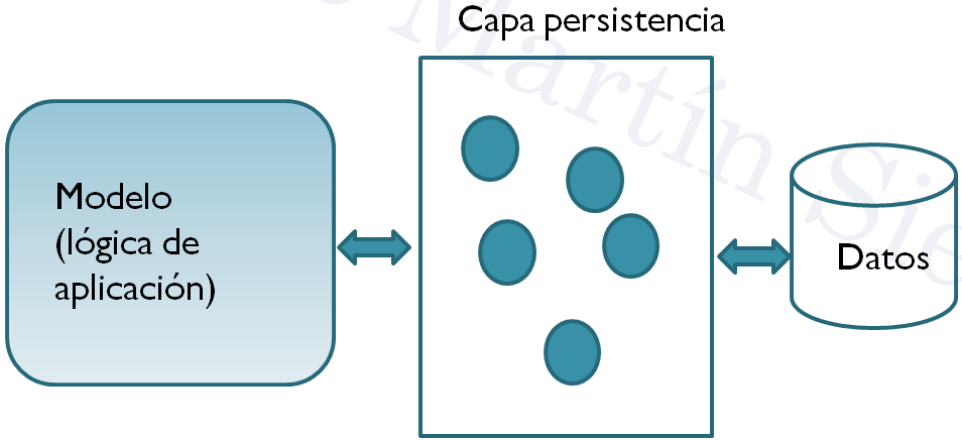
Antonio Martín Sierra

Java Persistence API



Capa de persistencia

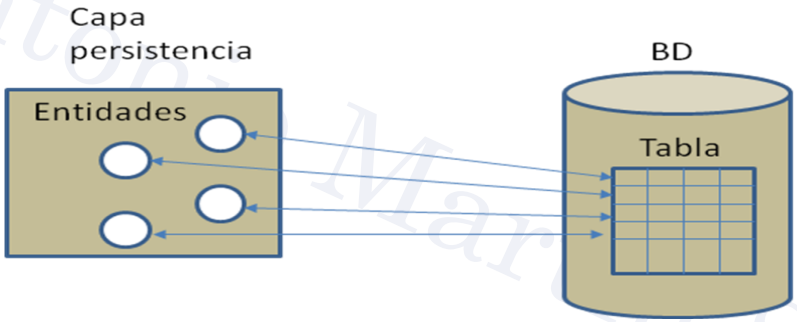
- Los datos son expuestos a la aplicación en forma de objetos (entidades).
- La lógica de negocio maneja objetos, no tablas



Antonio Martín Sierra

Entidades

- Objetos que representan una fila de una tabla de la base de datos.



- Se definen mediante clases tipo JavaBean

- Las operaciones sobre una capa de persistencia consisten en crear, modificar, eliminar y recuperar entidades.

Antonio Martín Sierra

Frameworks de persistencia

- Conjunto de utilidades que facilitan la creación y manipulación de una capa de persistencia.
- Entre los más populares:
  - Hibernate
  - Ibatis
  - JPA (no exactamente un framework)
- Componentes de un framework de persistencia
  - Motor de persistencia (mapeo ORM)
  - Sistema de configuración

Antonio Martín Sierra

Fundamentos JPA

- Especificación Java EE para la creación y gestión de una capa de persistencia
- Clases, interfaces y anotaciones definidas en el paquete javax.persistence
- Los principales frameworks de persistencia proporcionan una implementación de esta especificación

Antonio Martín Sierra

Componentes de JPA

- Archivo de configuración persistence.xml:
  - Define unidades de persistencia
  - Cada unidad de persistencia indica el motor utilizado, propiedades de conexión a BD y lista de entidades
- Conjunto de anotaciones para la configuración de entidades (@Entity, @Table, @Id,...)
- API JPA. Conjunto de clases e interfaces para acceso a la capa de persistencia

Antonio Martín Sierra

Creación de una capa de persistencia

- Creación de entidades y configuración a través de anotaciones
- Configuración de la unidad de persistencia a través del archivo persistence.xml, mediante clases de configuración Spring o archivo .properties/.yml en Spring Boot:
  - Proveedor de persistencia
  - Lista de entidades
  - Propiedades de conexión a base de datos

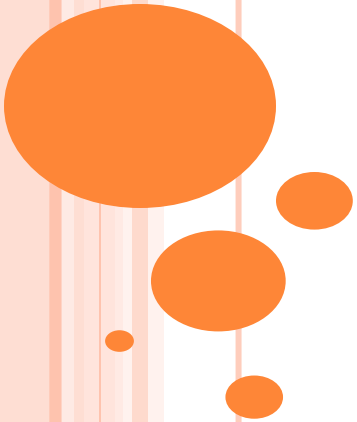
Antonio Martín Sierra

Principales anotaciones JPA

- ***@Entity*. Indica que la clase es una entidad**
- ***@Table*. Mapea la entidad a una tabla de la base de datos**
- ***@Id*. Indica el atributo que contiene la primary key**
- ***@Column*. Asocia el atributo con una columna de la tabla. No es necesario si el nombre del atributo coincide con el de la columna.**
- ***@GeneratedValue*. Indica la estrategia de generación de claves en columnas primary key autogeneradas**

Antonio Martín Sierra

Spring Data JPA



Spring Data JPA

- Elimina el código de acceso a los datos
- Definición de una subinterfaz de JpaRepository, que proporciona métodos CRUD

Tipo entidad

Tipo clave primaria

```
public interface AgendaSpringDao extends JpaRepository<Contacto, Integer>{

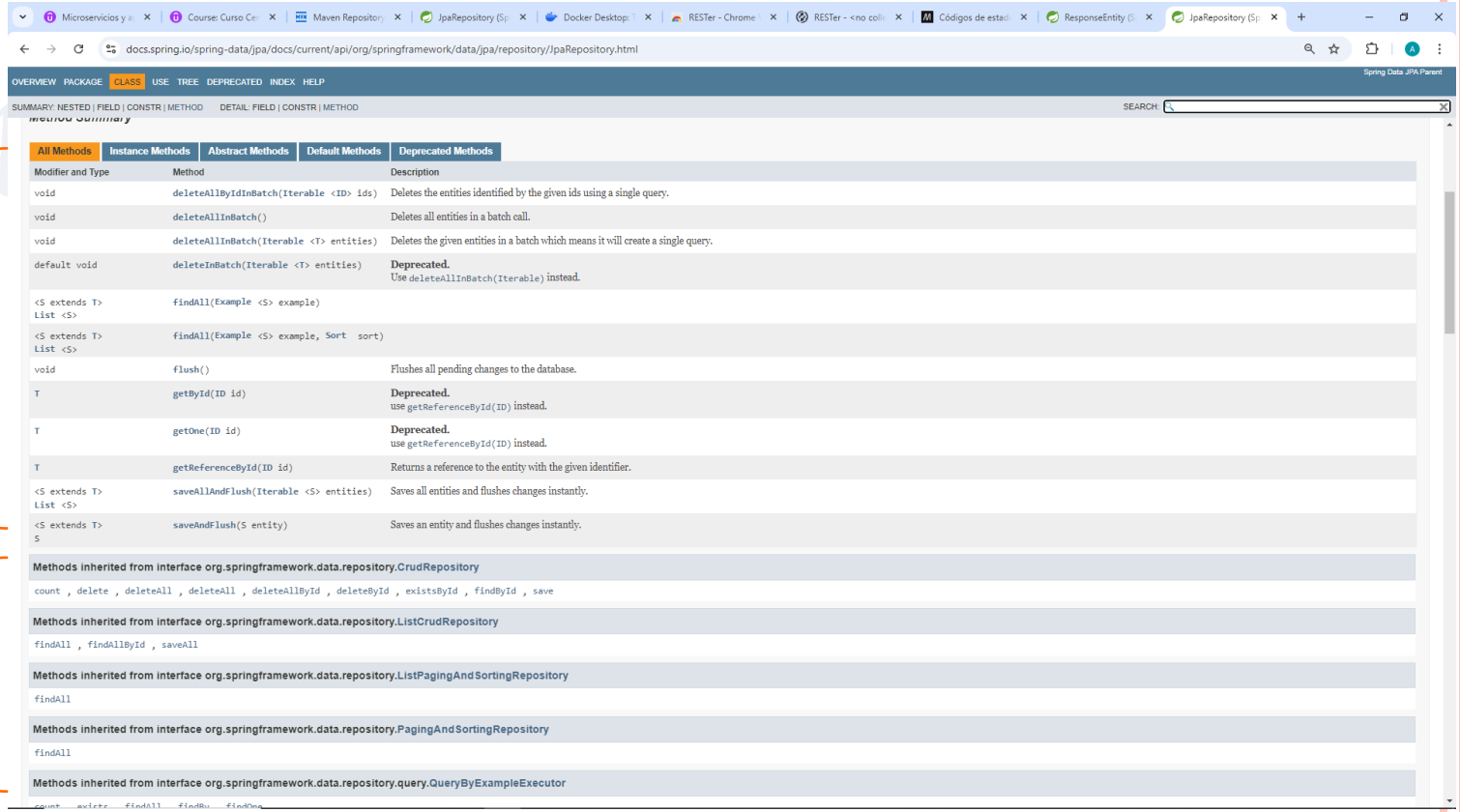
    ...
}
```

Antonio Manuel Olerra

JpaRepository

Métodos específicos JpaRepository

Métodos heredados



docs.spring.io/spring-data/jpa/docs/current/api/org/springframework/data/jpa/repository/JpaRepository.html

OVERVIEW PACKAGE CLASS USE TREE DEPRECATED INDEX HELP

Spring Data JPA Parent

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD SEARCH:

All Methods | Instance Methods | Abstract Methods | Default Methods | Deprecated Methods

Modifier and Type	Method	Description
void	deleteAllByIdInBatch(Iterable <ID> ids)	Deletes the entities identified by the given ids using a single query.
void	deleteAllInBatch()	Deletes all entities in a batch call.
void	deleteAllInBatch(Iterable <T> entities)	Deletes the given entities in a batch which means it will create a single query.
default void	deleteInBatch(Iterable <T> entities)	**Deprecated.** Use deleteAllInBatch(Iterable) instead.
<S extends T> List <S>	findAll(Example <S> example)	
<S extends T> List <S>	findAll(Example <S> example, Sort sort)	
void	flush()	Flushes all pending changes to the database.
T	getById(ID id)	**Deprecated.** use getReferenceById(ID) instead.
T	getOne(ID id)	**Deprecated.** use getReferenceById(ID) instead.
T	getReferenceById(ID id)	Returns a reference to the entity with the given identifier.
<S extends T> List <S>	saveAllAndFlush(Iterable <S> entities)	Saves all entities and flushes changes instantly.
<S extends T> S	saveAndFlush(S entity)	Saves an entity and flushes changes instantly.

Methods inherited from interface org.springframework.data.repository.CrudRepository

count , delete , deleteAll , deleteAll , deleteAllById , deleteById , existsById , findById , save

Methods inherited from interface org.springframework.data.repository.ListCrudRepository

findAll , findAllById , saveAll

Methods inherited from interface org.springframework.data.repository.ListPagingAndSortingRepository

findAll

Methods inherited from interface org.springframework.data.repository.PagingAndSortingRepository

findAll

Methods inherited from interface org.springframework.data.repository.query.QueryByExampleExecutor

Métodos JpaRepository

- Entre los principales métodos de JpaRepository:
  - T save(T entidad). Salva la entidad en la unidad de persistencia, si la entidad ya existe la actualiza. Devuelve la propia entidad
  - Optional<T> findById(ID id). Devuelve la entidad a partir de su primary key, envuelta en un Optional
  - List<T> findAll(). Recupera todas las entidades
  - void deleteById(ID id). Elimina la entidad a partir de su clave primaria

Antonio Martín Sierra

Métodos específicos

- **Spring implementa automáticamente métodos, a partir del nombre del mismo o instrucción JPQL:**

Implementación a partir del nombre de método

Implementación a partir de la query

```
public interface AgendaSpringDao extends JpaRepository<Contacto, Integer>{

    Contacto findByEmail(String email);

    @Query("Select c from Contacto c Where c.edad<=?1")
    List<Contacto> buscarPorEdadMaxima(int edad);

    @Transactional
    @Modifying
    @Query("Delete from Contacto c Where c.email=?1")
    void eliminarPorEmail(String email);
}
```

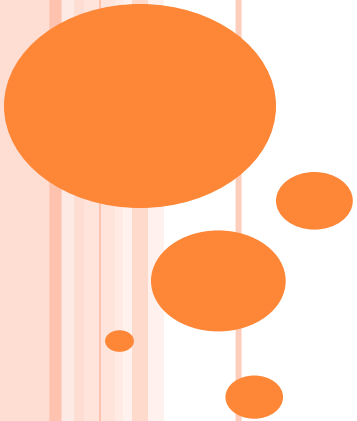
Query keywords

- Se trata de palabras reservadas que pueden utilizarse a la hora de definir el nombre de un método autogenerado:
  - findBy.. . Se utiliza para buscar entidades por una o varias propiedades
  - findFirstBy.. . Utilizado cuando la consulta debe entregar un único resultado
  - streamBy.. .Retorna un Stream de entidades, mejorando el rendimiento con alto volumen de datos
  - deleteBy.. .Para eliminar por alguna propiedad
  - distinctBy.. . Recupera entidades que sean distintas por alguna propiedad
  - existsBy.. .Devuelve boolean indicando si existe entidad por criterio
- Se pueden usar además los operadores: And, Or, Between, IsNot, LessThan, GreaterThan, Like, In, NotIn:

findByAgeIn(List<Integer> ages)

Antonio Martín Sierra

Documentacion de servicios con Spring doc



Spring doc

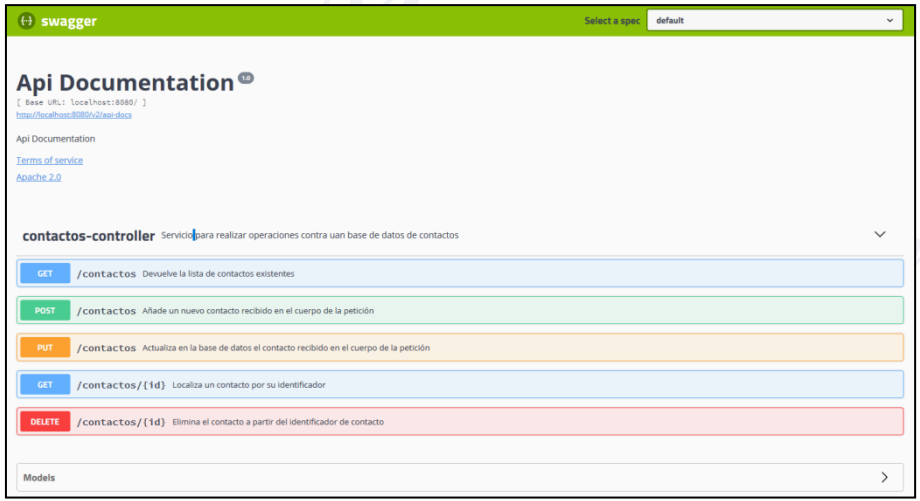
- **Librería Spring para la documentación de servicios REST bajo el estándar Open API.**
- **Open API: Estándar abierto para la documentación de servicios REST. Admite diferentes formatos, entre ellos swagger.**
- **Aunque se puede utilizar con versiones anteriores de Spring Boot, *desde Spring Boot 3.x es la única librería compatible para la documentación de servicios REST*.**

Antonio Martín Olivera

El estilo swagger

- Swagger es la herramienta utilizada tradicionalmente para generar la documentación de un servicio REST.

- Spring doc permite generar documentación siguiendo el estilo clásico de swagger:



Dependencias Spring doc

- **Para documentar servicios con Spring doc a partir de Spring Boot 3.x, se emplean la siguiente dependencias:**

```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.8.13</version>
</dependency>
```

- **No es necesario definir ninguna configuración especial para generar una documentación estilo swagger**

Anotaciones del controlador

- **Spring doc soporta las siguientes anotaciones para la definición de la clase controladora:**

  - **@Operation. Utilizada en cada operación expuesta por el controller. El texto inicial de ayuda se indica a través de *summary*, mientras que en *description* se ofrece información más detallada.**

  - **@Parameter. Se utiliza delante de cada parámetro del método controlador para informar sobre el uso del mismo.**

  - **@ApiResponse. Con el atributo *responseCode* indica uno de los posibles códigos de estado devueltos como respuesta. Mediante el atributo *description*, se puede proporcionar información detallada sobre el motivo de esta respuesta. Para agrupar varias posibles respuestas, se incluirían dentro de la anotación @ApiResponses.**

Antonio Martín Sierra

Ejemplo controlador

```
@Operation(summary = "Lista de items por temática",
            description = "A partir de la temática recibida como prámetro, devuelve los items correspondientes")
@GetMapping(value="items",produces=MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<List<ItemDto>> buscarPorTematica(@Parameter(description = "Nombre de la temática de búsqueda")
                                                        @RequestParam String tematica){
   return new ResponseEntity<>(itemsService.buscarPorTematica(tematica),HttpStatus.OK);
}

@ApiResponses({
   @ApiResponse(responseCode = "201", description = "Alumno creado correctamente", content = @Content),
   @ApiResponse(responseCode = "409", description = "No se pudo crear el alumno (conflicto)", content = @Content)
})
@PostMapping(value="items", consumes=MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<Void> altaItem(@RequestBody ItemDto item) {
   if(itemsService.nuevoItem(item)) {
      return new ResponseEntity<>(HttpStatus.CREATED);
   }
   return new ResponseEntity<>(HttpStatus.CONFLICT);
}
```

Acceso a la ayuda

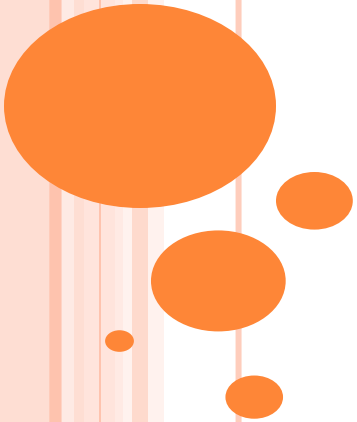
- **Al compilar y ejecutar el servicio, se podrá acceder a la ayuda del mismo a través de la dirección:**

http://dir_ip:puerto/context-path/swagger-ui.html

Antonio Martín Sierra



Testing de microservicios



Testing

- Permite probar el funcionamiento de cada capa de la aplicación de forma independiente

- Spring Boot incluye el starter para poder crear clases de test:

```
<!-- canonical test starter providing MockBean, WebMvcTest, Jackson, Mockito, JUnit -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

- Para testing de la capa de repositorio se debe incluir H2 para generar los mocks de datos

Antonio Sierra

Testing capa repository

- Se prueba el correcto funcionamiento de los métodos definidos en la interfaz JpaRepository
- La anotación @DataJpaTest genera el proceso

```
@DataJpaTest
public class ItemsRepositoryTest {
  @Autowired
  ItemsRepository itemsRepository;
  @Test
  void testFindByTematica() {
    itemsRepository.deleteAll();
    itemsRepository.save(new Item(null,"url1", "java", "desc1"));
    itemsRepository.save(new Item(null,"url2", "java", "desc2"));
    itemsRepository.save(new Item(null,"url3", "spring", "desc3"));
    List<Item> javaItems = itemsRepository.findByTematica("java");
    List<Item> pythonItems = itemsRepository.findByTematica("python");
    assertThat(javaItems).hasSize(2)
      .extracting(Item::getUrl)
      .containsExactlyInAnyOrder("url1", "url2");
    assertThat(pythonItems).hasSize(0);
  }
  @Test
  void testFindFirstByUrl() {
    itemsRepository.deleteAll();
    itemsRepository.save(new Item(null, "unique-url", "testing", "desc"));
    Item found = itemsRepository.findFirstByUrl("unique-url");
    assertThat(found).isNotNull();
    assertThat(found.getTematica()).isEqualTo("testing");
  }
```

Testing capa service

➢Se utiliza Junit con Mockito

```
@ExtendWith(MockitoExtension.class)
public class TestItemsService {
    @Mock
    ItemsRepository itemsRepository;
    @Mock
    ItemMapper itemMapper;
    @InjectMocks
    ItemsServiceImpl itemsService;
    @Test
    void testNumevoItem() {
        Item item = new Item(0, "unique-url", "testing", "desc");
        ItemDto dto = new ItemDto(0, "unique-url", "testing", "desc");
        when(itemMapper.toEntity(dto)).thenReturn(item);
        when(itemsRepository.existsByUrl("unique-url")).thenReturn(false);
        when(itemsRepository.save(any(Item.class))).thenReturn(item);
        boolean created = itemsService.nuevoItem(dto);
        assertThat(created).isTrue();
        verify(itemMapper).toEntity(dto);
        verify(itemsRepository).existsByUrl("unique-url");
        verify(itemsRepository).save(item);
    }
}
```

Sierra

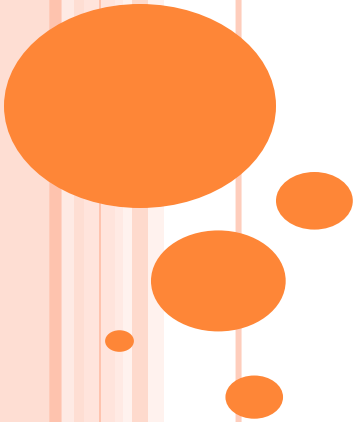
Testing capa controller

- **Se utiliza Junit, Mockito y WebMvcTest**

```
@WebMvcTest(ItemsController.class)
public class TestItemsController {
   @MockitoBean
   private ItemsService itemsService;
   @Autowired
   private MockMvc mockMvc;
   private ObjectMapper objectMapper = new ObjectMapper();
   @Test
   void testBuscarPorTematica() throws Exception {
      List<ItemDto> items = Arrays.asList(
            new ItemDto(1, "u1", "java", "d1"),
            new ItemDto(2, "u2", "java", "d2")
         );

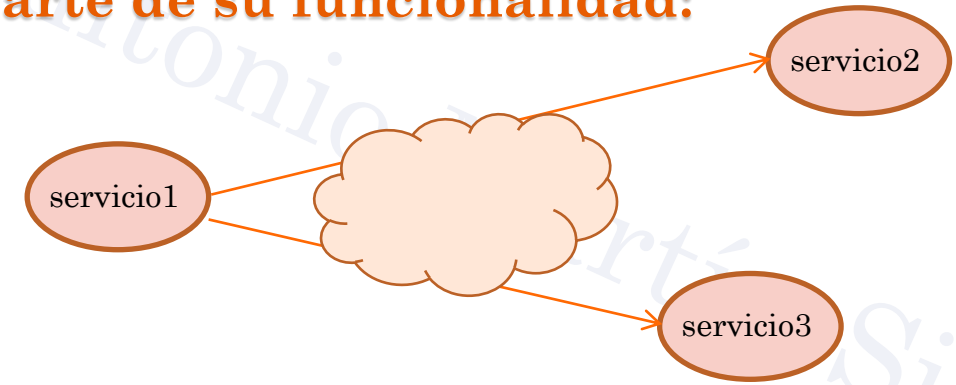
      when(itemsService.buscarPorTematica("java")).thenReturn(items);
      mockMvc.perform(get("/v1/items").param("tematica", "java"))
         .andExpect(status().isOk())
         .andExpect(jsonPath("$.length()").value(2))
         .andExpect(jsonPath("$[0].url").value("u1"));
   }
   @BeforeEach
   void setup() {
      mockMvc = MockMvcBuilders.standaloneSetup(itemsController).build();
   }
}
```

Interacción entre microservicios



Escenario

- En muchos escenarios, un microservicio necesita llamar a otro servicio (puede ser o no un microservicio) REST, como parte de su funcionalidad:



- Además de las ya conocidas RestTemplate y WebClient, en Spring 6.1 (Spring Boot 3.2), se ha incorporado la interfaz RestClient para llamar a un servicio REST desde otra aplicación Spring

Antonio J. Sierra

La interfaz RestClient

- Forma parte del módulo Web MVC de Spring, incluido en el starter Web, por lo que no habrá que añadir ningún otro starter adicional.

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

- Características:
  - Muy similar a WebClient
  - Estilo de programación funcional
  - Exclusivamente para peticiones síncronas

Arturo Martín Sierra

Creación de un objeto RestClient

- **Definimos un método en la clase de configuración para su creación mediante el método estático *create()* de la propia interfaz:**

```
@Bean
public RestClient getRestClient() {
        return RestClient.create();
}
```

- **Para inyectarlo en una variable se emplea la anotación @Autowired de Spring:**

```
@Autowired
RestClient restClient;
```

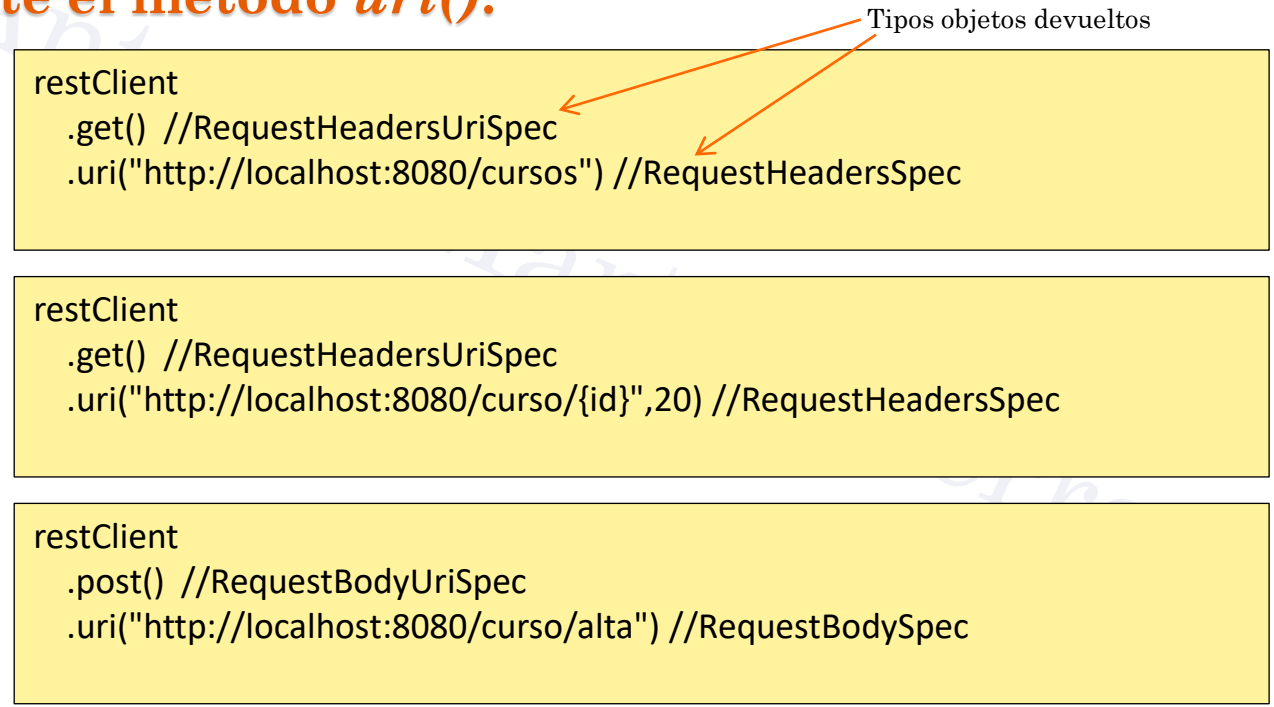
Configuración método de petición

- **RestClient proporciona una serie de métodos para establecer el método de petición Http.**
- **Cada método devuelve un objeto para continuar configurando la petición:**
  - **RequestHeadersUriSpec *get()***
  - **RequestHeadersUriSpec *delete()***
  - **RequestBodyUriSpec *post()***
  - **RequestBodyUriSpec *put()***

Antonio Martín Sierra

Dirección de la petición

- **A partir de los objetos devueltos por los métodos anteriores se puede configurar la dirección de la petición mediante el método *uri()*:**



Cuerpo de la petición

- **En peticiones POST y PUT se puede enviar un objeto en la petición, habitualmente en formato JSON.**

- **Se configura a través de los métodos *contentType()* y *body* de RequestBodySpec:**

```
webClient
   .post()  //RequestBodyUriSpec
   .uri("http://localhost:8080/curso/alta") //RequestBodySpec
   .contentType(MediaType.APPLICATION_JSON) //RequestBodySpec
   .body(new Curso(…)) //RequestHeadersSpec
```

- **Opcionalmente, se pueden establecer encabezados a través del método *header()* de RequestHeadersSpec**

Lanzamiento de la petición

- **Una vez configurada la uri, encabezados y cuerpo, se procede a lanzar la petición a través del método *retrieve()* de RequestHeadersSpec.**

- **El método devuelve un ResponseSpec para definir la forma en la que se extraerá la respuesta**

```
restClient
   .get()  //RequestHeadersUriSpec
   .uri("http://localhost:8080/curso/{id}",20) //RequestHeadersSpec
   .retrieve() //ResponseSpec
```

Antonio Martínez

Extracción de la respuesta

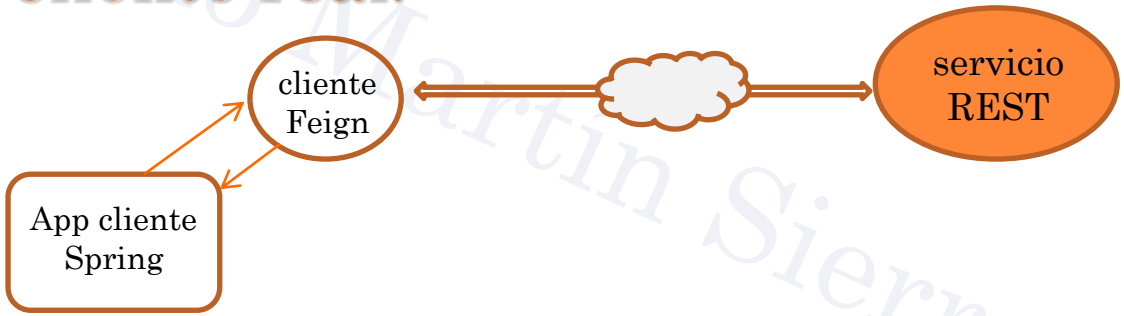
- **Mediante *body()* de ResponseSpec se extrae directamente el cuerpo de la respuesta, indicando el tipo al que debe convertirse dicho cuerpo.**

```
Curso curso=restClient
                .get()  //RequestHeadersUriSpec
                .uri("http://localhost:8080/curso/{id}",20) //RequestHeadersSpec
                .retrieve() //ResponseSpec
                .body (Curso.class); //Curso
```

- **Otros métodos permiten obtener la respuesta como un ResponseEntity:**
  - **ResponseEntity<Void> toBodilessEntity()**
  - **ResponseEntity<T> toEntity(class<T> bodyType)**

Cliente Feign

- Cliente de un servicio REST cuya implementación es proporcionada por Spring
- Actual de proxy entre el servicio REST y la aplicación cliente real:



- Compatible con Eureka

Antonio Martín Sierra

Dependencias y configuración

- **En la aplicación o microservicio cliente añadiremos el siguiente starter:**

```
<dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

- **Se añadirá la anotación *@EnableFeignClients* en la clase lanzadora:**

```
@EnableFeignClients
@ComponentScan(basePackages= …)
@SpringBootApplication
public class Application {
   …
}
```

Definición de la interfaz

- **Para que spring implemente el cliente Feign, es necesario definir una interfaz con los métodos que debe exponer:**

Identificador del servicio remoto. Si se quiere acceder directamente en lugar de hacerlo por eureka, se utilizará el atributo url

Configura las peticiones que serán lanzadas desde los métodos

```
@FeignClient(value="servicio-contactos")
public interface FeignContactos {
   @GetMapping(value="contactos",produces = MediaType.APPLICATION_JSON_VALUE)
   List<Persona> getPersonas();

   @PostMapping(value="contactos",produces = MediaType.APPLICATION_JSON_VALUE,
                                          consumes =    MediaType.APPLICATION_JSON_VALUE)
   List<Persona> altaPersona(@RequestBody Persona pe);
}
```

Antonio M

Http Client Interfaces

- Mecanismo introducido en Spring 4 para simplificar el acceso a servicios remotos
- Similar al antiguo OpenFeign
- Spring genera un proxy a partir de una interfaz Java.
- La aplicación cliente accede al servicio remoto a través de ese proxy, como si se tratase de un objeto local
- El cliente se abstrae de los detalles de acceso al servicio

Antonio Martín Sierra

Definición de la interfaz

- Se crea una interfaz en la que se definen los métodos que debe implementar el proxy.
- A través de anotaciones, se indica a Spring las direcciones de los recursos remotos, parámetros, etc.

Métodos HTTP que el proxy debe utilizar en la llamada

```
public interface LibrosClient {
    @GetExchange("libros")
    List<Ejemplar> libros();
    @PostExchange("libros")
    void alta(@RequestBody Ejemplar ejemplar);
}
```

Utilización

- **En la capa de negocio se le pide a Spring que inyecte el proxy y se utiliza como un objeto normal. No tenemos que indicar ningún detalle relativo a la comunicación:**

```
@Service
public class EjemplaresServiceImpl implements EjemplaresService {
    LibrosClient librosClient;
    public EjemplaresServiceImpl(LibrosClient librosClient) {
        this.librosClient = librosClient;
    }
    @Override
    public void altaEjemplar(Ejemplar ejemplar) {
        try {
            librosClient.alta(ejemplar);
        }catch(Exception ex) {
            throw new EjemplarExistenteException();
        }
    }
    @Override
    public List<Ejemplar> ejemplaresTematica(String tematica) {
        return
                librosClient.libros().stream()
                .filter(e->e.getTematica().equals(tematica))
                .toList();
    }
}
```

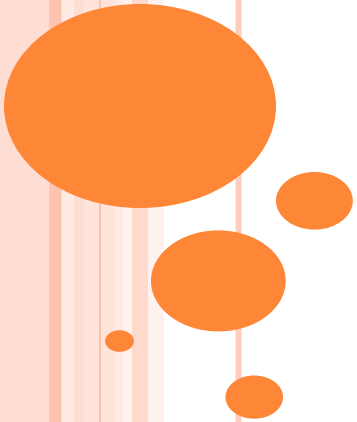
Configuración

- **En la clase de configuración se define la creación del RestClient y del objeto proxy:**

```
@Bean
RestClient restClient() {
    return RestClient.create("http://localhost:8500");
}
@Bean
LibrosClient librosClient(RestClient restClient) {
    HttpServiceProxyFactory factory =
        HttpServiceProxyFactory
            .builderFor(
                RestClientAdapter.create(restClient))
            .build();

    return factory.createClient(LibrosClient.class);
}
```

Securización de microservicios



Concepto

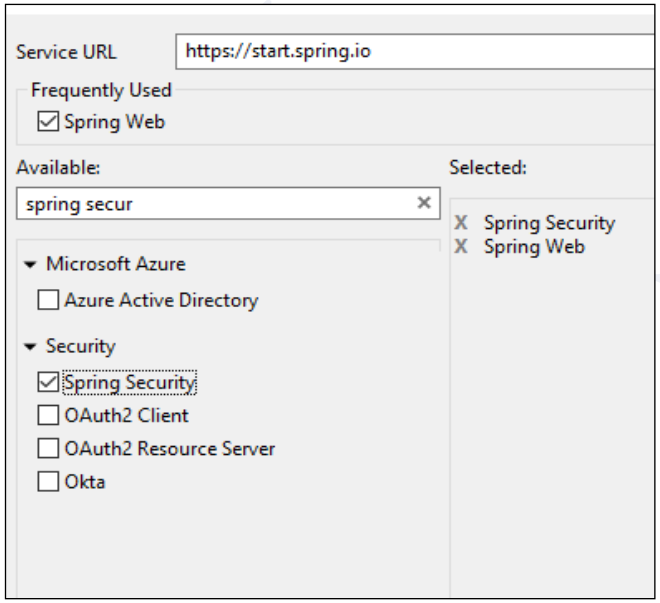
- Securizar un servicio consiste en permitir el acceso a los recursos, solamente a usuarios autorizados
- Se requiere un proceso de autenticación y autorización
- Gestionado por Spring Security

Antonio Martín Sierra



Dependencias

- Se debe incorporar el starter de spring security




```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Martin Sierra

Configuración

- **Se establece a través de dos métodos en una clase de configuración:**

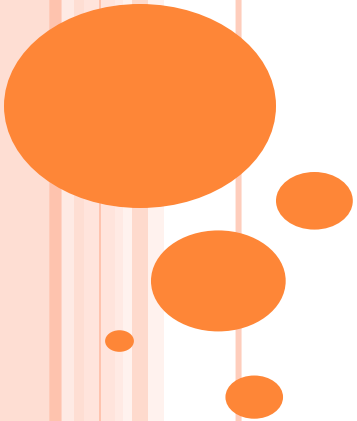
Definición de roles y usuarios

Políticas de acceso a recursos

```
@EnableWebSecurity
@Configuration
public class SecurityConfig {
   @Bean
   public InMemoryUserDetailsManager usersDetails () throws Exception {
       List<UserDetails> users=List.of(
                   User.withUsername("user1")
                           .password("{noop}user1")
                           .roles("USER")
                           .build(),
                    …);
        return new InMemoryUserDetailsManager(users);
   }
   @Bean
   public SecurityFilterChain  filterChain(HttpSecurity http) throws Exception {
     ..
     return http.build();
   }

}
```

Securización de microservicios mediante token JWT

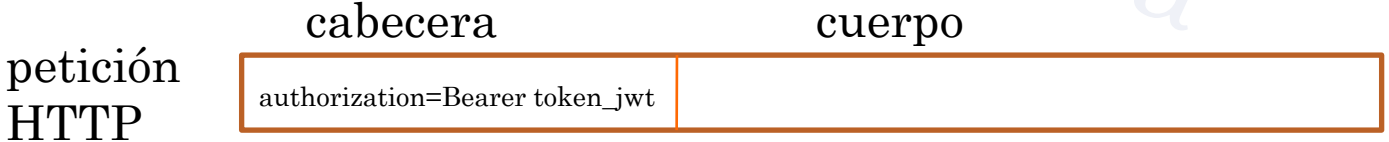


¿Qué es un token JWT?

- **Mecanismo para comunicar de forma segura un microservicio con una aplicación cliente y proceder a su identificación.**

- **Se basa en el uso de una cadena JSON codificada y firmada, que incluye los datos del cliente (usuario, roles,..).**

- **El cliente envía el token JWT al microservicio en la cabecera de las peticiones, éste la decodifica y verifica que no ha sido alterada en tránsito, extrayendo a continuación la información del cliente para proceder a su autorización.**



Antonio Martín Sierra

Estructura de un token JWT

- **Un token JWT es una cadena codificada que consta de tres partes separadas por un ".":**

header . payload . signature

header
```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

payload
```
{
  "iat": 1632173019,
  "sub": "admin",
  "authorities": [
    "ROLE_ADMIN",
    "ROLE_USER"
  ],
  "exp": 1632259419
}
```

signature
```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  secret_key
)
```

header: Incluye dos campos, el tipo de token (siempre JWT) y el mecanismo de firma

payload: Contiene la información del cliente, como el nombre, roles, etc. También el tiempo de vida del token

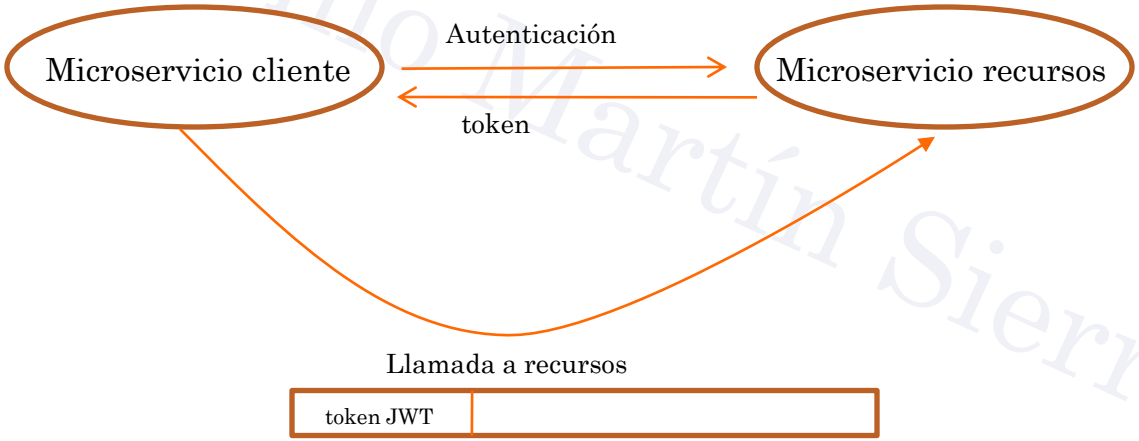
signature. Se construye utilizando la función especificada en la cabecera y que recibe como parámetro la cabecera más el payload codificado, además de la clave secreta preestablecida. Esto permite verificar la integridad del mensaje

Antonio Martín Sierra

eyJhbGciOiJIUzUxMiJ9
.eyJpYXQiOjE2MzQxNDI0ODgsInN1YiI6ImFkbWluIiwiYXV0aG9yaXRpZXMiOlsiUk9MRV9BRE1JTiIsIlJPTEVfVVNFUiJdLCJleHAiOjE2MzQyMjg4ODh9
.JsiPFDtOjkCW5AwwmloE3cz_WTl3ZM9CC4_ZKw7XbBH-zEedc0-geuYEc3BIWM3Yf7dtZQA3c5mmToRizIJk6g

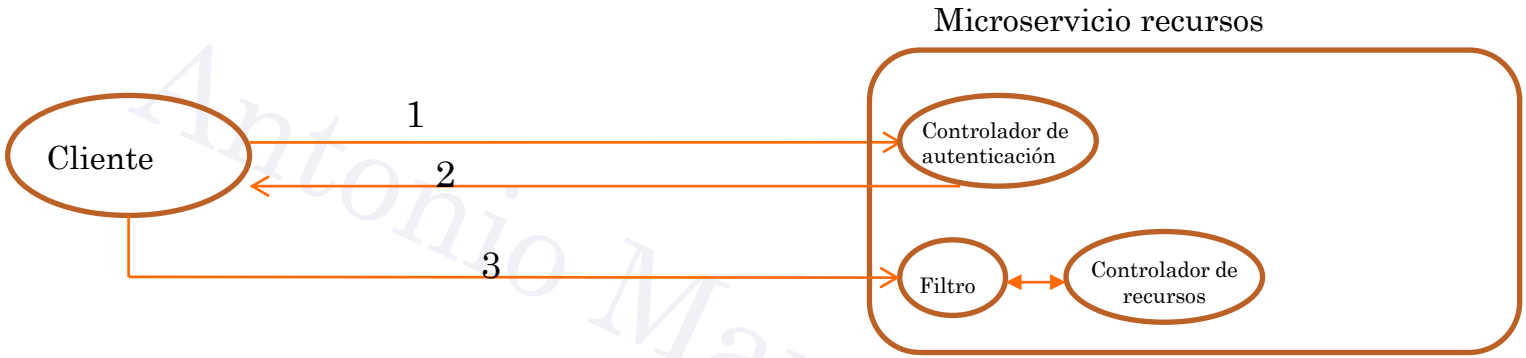
Token JWT y microservicios

- El microservicio cliente se autentica previamente en el microservicio que quiere utilizar para obtener el token.
- El cliente utiliza el token para acceder a los recursos



Antonio Martín Sierra

Microservicio de recursos



Antonio Martín Sierra

1.-El cliente se autentica a través de un controlador específico
2.-El controlador genera el token y se lo envía al cliente
3.-El cliente presenta el token en cada petición y un filtro se encarga de su validación

Dependencias

- **Además del starter de spring security, se requiere la librería jjwt encargada de generar y decodificar el token:**

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.1</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.11.1</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.11.1</version>
    <scope>runtime</scope>
</dependency>
```

Autenticación y generación del token

- **Se deberá implementar un controller específico encargado de la autenticación y generación del token**

```
@RestController
public class AuthController {
    AuthenticationManager authManager;
    public AuthController(AuthenticationManager authManager) {
        this.authManager=authManager;
    }
    @PostMapping("login")
    public ResponseEntity<String> login(@RequestParam("user") String user, @RequestParam("pwd") String pwd) {
        try {
            Authentication autentication=authManager.authenticate(new UsernamePasswordAuthenticationToken(user,pwd));
            return new ResponseEntity<>(getToken(autentication),HttpStatus.OK);
        }catch(AuthenticationException ex) {
            ex.printStackTrace();
            //si el usuario no es válido, enviamos el código de estado 401
            return new ResponseEntity<>("No autorizado",HttpStatus.UNAUTHORIZED);
        }
    }
    private String getToken(Authentication autentication) {
        //generación del token a partir de los credenciales
    }
}
```

Configuración de seguridad

- **La configuración de spring security permanece intacta, únicamente habrá que indicar el filtro encargado de verificar el token:**

```
@EnableWebSecurity
@Configuration
public class SecurityConfig {
    AuthenticationManager auth;
    @Bean
    public AuthenticationManager authManager(AuthenticationConfiguration conf) throws Exception{
        auth=conf.getAuthenticationManager();
        return auth;
    }
    @Bean
    public JdbcUserDetailsManager usersDetailsJdbc() {
        :
    }
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception{
        http.csrf(cus->cus.disable())
        .authorizeHttpRequests(aut->
            aut.requestMatchers(HttpMethod.POST,"/contactos").hasRole("ADMIN")
            .requestMatchers(HttpMethod.DELETE,"/contactos/**").hasAnyRole("ADMIN","OPERATOR")
            .requestMatchers("/contactos").authenticated()
            .anyRequest().permitAll()
            )
        .addFilter(new JWTAuthorizationFilter(auth));
        return http.build();
    }
}
```

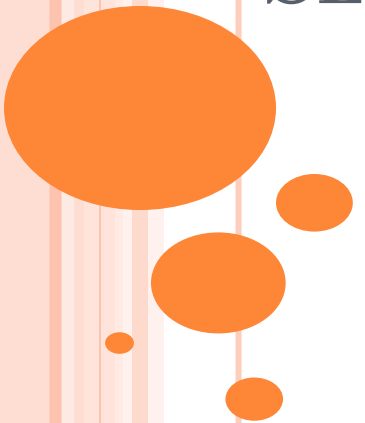
Filtro de seguridad

Filtro de seguridad

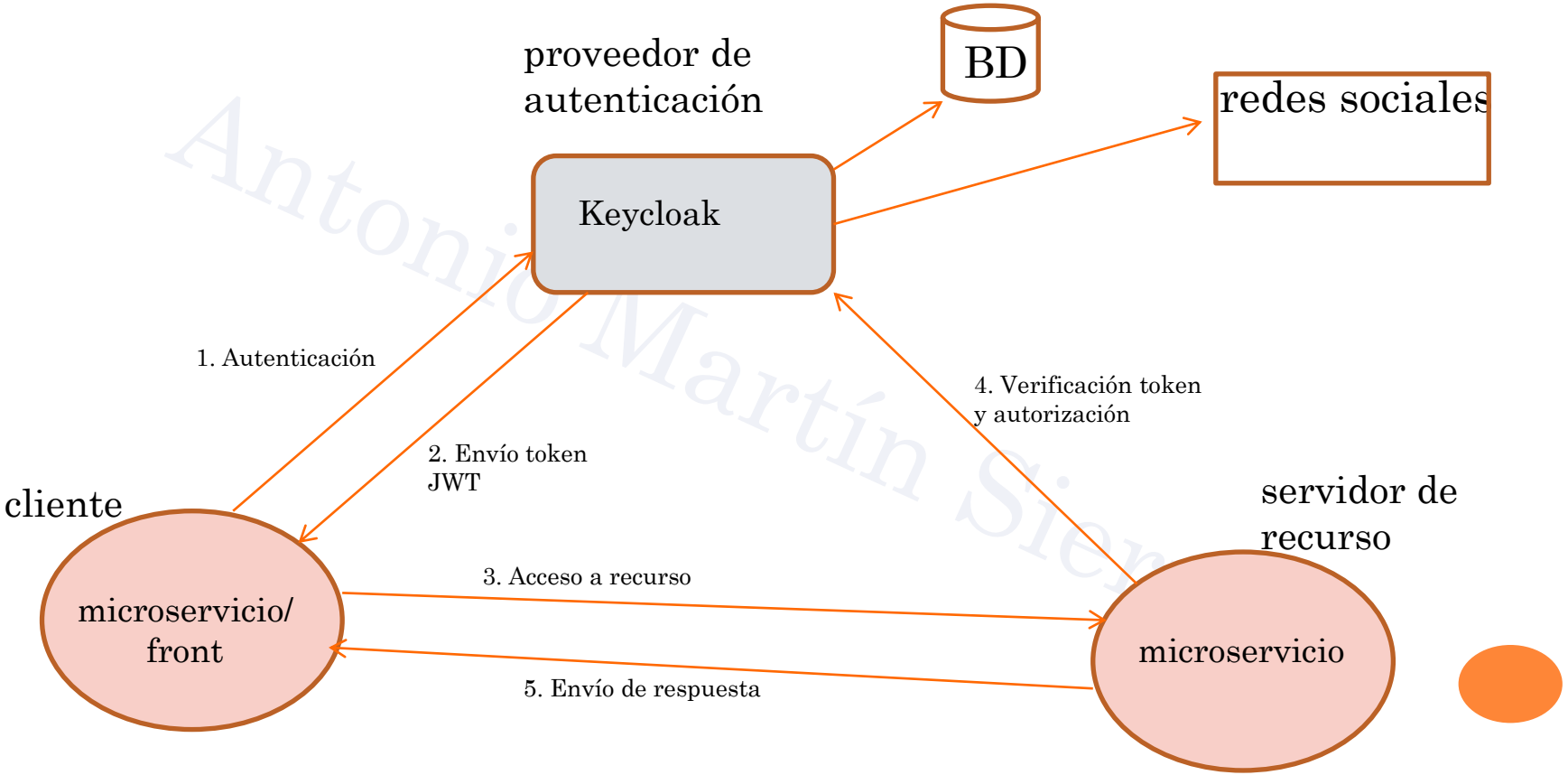
- **Subclase de BasicAuthenticationFilter. En doInternalFilter() se decodifica el token y se obtiene la información del usuario, pasando el control a Spring Security para que proceda a la autorización:**

```
public class JWTAuthorizationFilter extends BasicAuthenticationFilter {
    public JWTAuthorizationFilter(AuthenticationManager authManager) {
        super(authManager);
    }
    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
            throws IOException, ServletException {
        String header = req.getHeader(ENCABEZADO);
        if (header == null || !header.startsWith(PREFIJO_TOKEN)) {
            chain.doFilter(req, res);
            return;
        }
        //obtenemos los datos del usuario a partir del token
        UsernamePasswordAuthenticationToken authentication = getAuthentication(req);
        //la información del usuario se almacena en el contexto de seguridad
        //para que pueda ser utilizada por Spring security durante el
        //proceso de autorización
        SecurityContextHolder.getContext().setAuthentication(authentication);
        chain.doFilter(req, res);
    }
    private UsernamePasswordAuthenticationToken getAuthentication(HttpServletRequest request) {
        //obtención de datos del usuario a partir del token
    }
}
```

Securización mediante OAuth2



Esquema OAuth2



Antonio Martín Sier

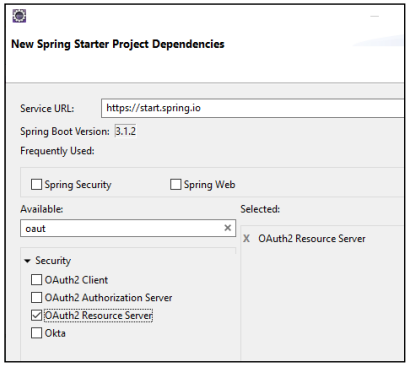
Gestor de identidades Keycloak

- Gestor de identidades Open Source.
- Funciona como proveedor de autenticación para aplicaciones. Las aplicaciones delegan la autenticación de usuarios en Keycloak
- Genera token JWT
- Integración con proveedores de autenticación externos (redes sociales,...)

Antonio Martín Sierra

OAuth2 en servidor de recurso

- El servidor de recurso necesita comunicar con Keycloak para la verificación del token JWT.
- Además de Spring Security requiere el starter oauth2 resource server:




```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

Antonio M. Sierra

Propiedades configuración

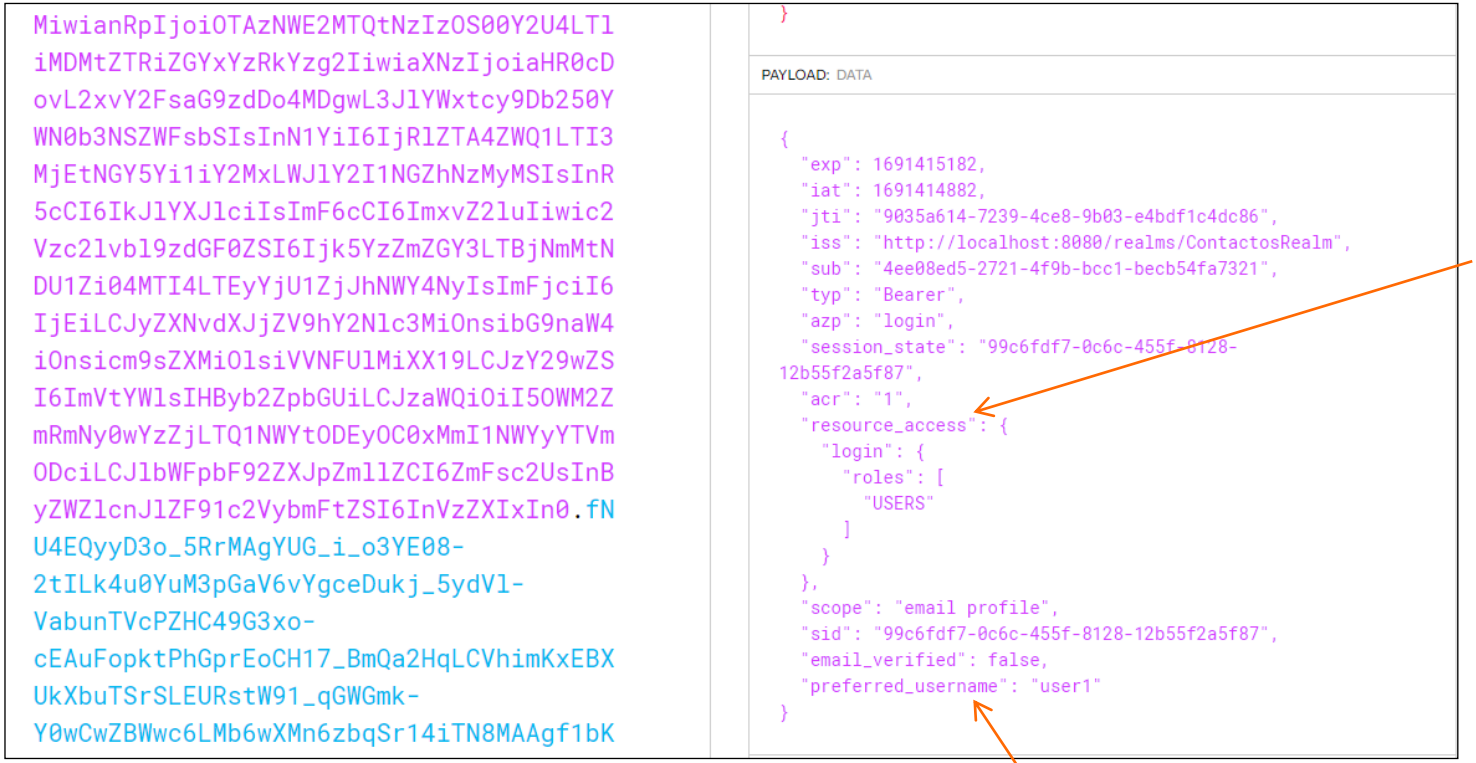
- **En el archivo .properties se deben definir una serie de propiedades para que el servidor de recurso pueda validar el token, como los datos de conexión al reino de Keycloak y la clave de la firma:**

spring.security.oauth2.resourceserver.jwt.issuer-uri=
http://localhost:8080/realms/ContactosRealms
spring.security.oauth2.resourceserver.jwt.jwk-set-uri=
${spring.security.oauth2.resourceserver.jwt.issuer-uri}/protocol/openid-connect/certs

- **Propiedades personalizadas adicionales:**

keycloak.clientId=login

Estructura Token JWT



Conversor

➢Componente utilizado para extraer la información del token:

Tipo entrada

Tipo salida

Inyección propiedades personalizadas

```
@Component
public class JwtAuthConverter implements Converter<Jwt, AbstractAuthenticationToken> {
    @Value("${keycloak.clientId}")
    private String clientId;

    @Override
    public AbstractAuthenticationToken convert(Jwt jwt) {
     …
    }
}
```

Genera un objeto con los datos del usuario extraídos del token

Configuración Spring security

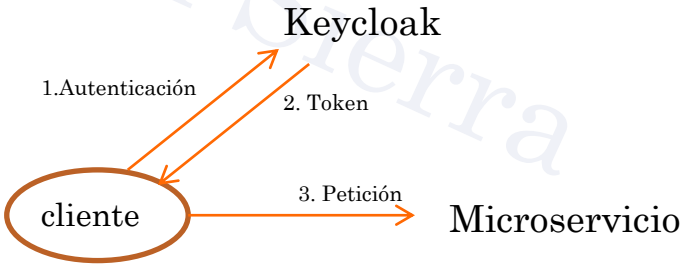
- **Mantiene la configuración de autorización.**
- **En la configuración para OAuth2, se indica el conversor utilizado:**

Inyección componente conversor

```
@EnableWebSecurity
@Configuration
public class SecurityConfig {
   @Autowired
   private  JwtAuthConverter jwtAuthConverter;
   @Bean
   public SecurityFilterChain filterChain(HttpSecurity http) throws Exception{
      http.csrf(cus->cus.disable())
      .authorizeHttpRequests(aut->
         aut.requestMatchers(HttpMethod.POST,"/contactos").hasRole("ADMIN")
         .requestMatchers(HttpMethod.DELETE,"/contactos/**").hasAnyRole("ADMIN","OPERATOR")
         .requestMatchers("/contactos").authenticated()
         .anyRequest().permitAll())
      .oauth2ResourceServer(oauth2ResourceServer->
         oauth2ResourceServer.jwt(jwt->jwt
            .jwtAuthenticationConverter(jwtAuthConverter)))
            .sessionManagement(sessionManagement->
                sessionManagement.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
      return http.build();
   }
}
```

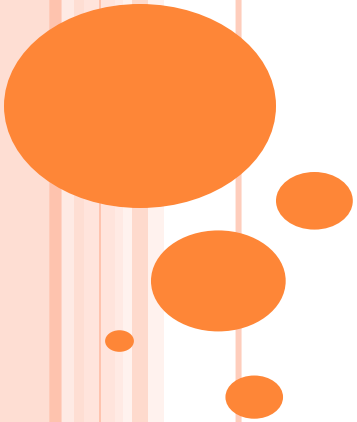
OAuth2 en el cliente

- El cliente no requiere ninguna dependencia ni configuración especial.
- Debe encargar de lanzar la petición post a Keycloak con los datos para realizar el proceso de autenticación y obtención del token
- Una vez recibido el token, lo incluirá en las cabeceras de las peticiones a microservicio



Antonio Martín Sierra

Acceso a un servicio seguro mediante RestClient



Autenticación básica

- **Se deben incluir credenciales en la cabecera de petición Authorization**
- **El formato del encabezado será:**

Basic ery45ju=

base64(usuario:password)

- **La cabecera se incluye mediante método *header()* de RequestHeadersSpec:**

```
.header("Authorization","Basic ery45ju=");
```

Antonio Martín Sierra

Autenticación con token

- **De la misma forma que antes, el token se incluye en la cabecera de petición Authorization**
- **El formato del encabezado será:**

Bearer d7es.jh3h4hh2ffed

token JWT

- **La cabecera se incluye mediante método *header()* de RequestHeadersSpec:**

```
.header("Authorization","Bearer d7es.jh3h4hh2ffed");
```

Antonio Martín Sierra

Perfiles

Definición y Objetivo

- **A través de los perfiles podemos crear diferentes configuraciones para un mismo proyecto.**
- **Esto permite adaptar nuestro proyecto a entornos diferentes.**
- **Cuando vamos a cambiar de entorno, simplemente elegimos la configuración adecuada, evitando realizar continuas modificaciones en archivos .properties o .yml**
- **Los archivos de configuración de cada entorno serán nombrados:**

application-entorno1.properties
application-entorno2.properties

Antonio Martín Sierra

Elección de un entorno

- **Para elegir un determinado archivo de configuración antes de empaquetar o ejecutar nuestro proyecto, configuramos la siguiente propiedad en el archivo application.properties:**

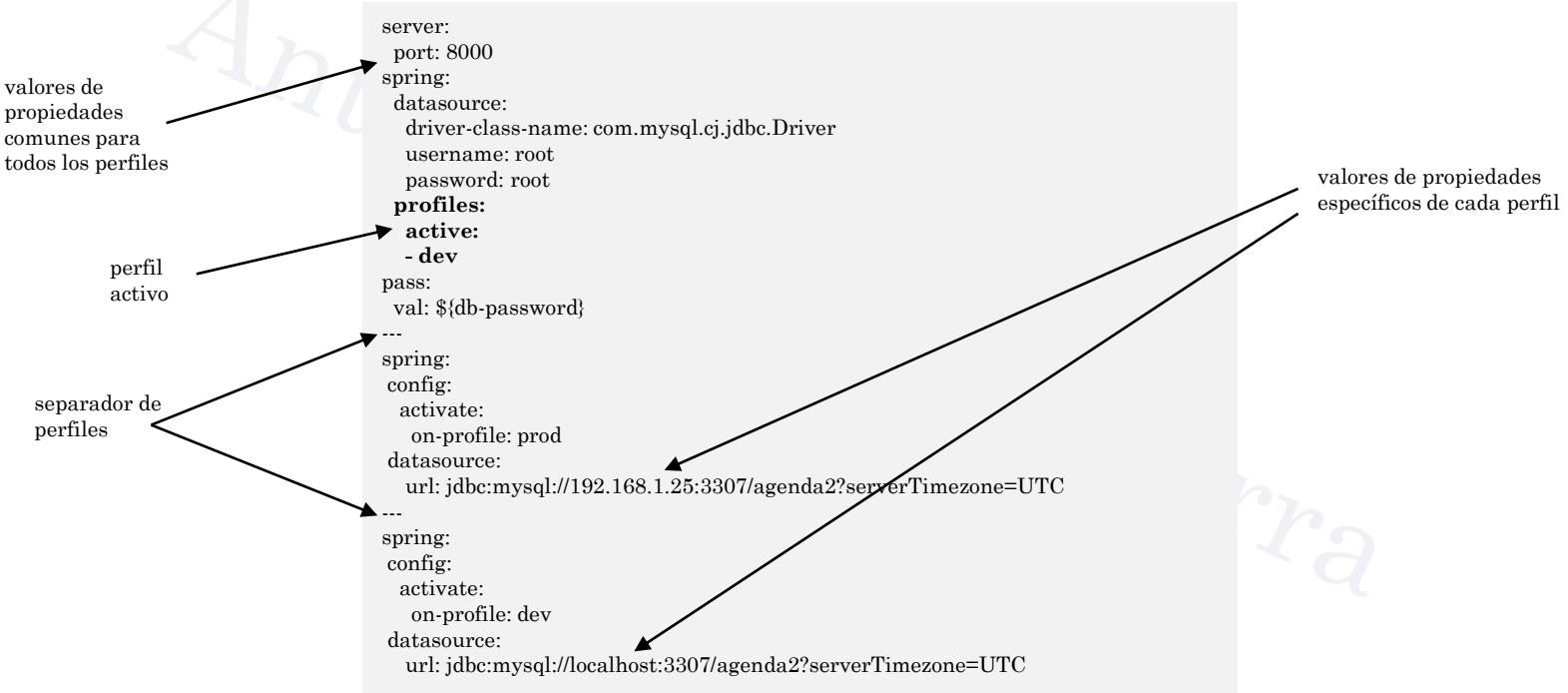
```
spring.profiles.active=entorno1
```

- **El archivo principal application.properties puede incluir también aquellas propiedades cuyos valores sean comunes a todos los perfiles**

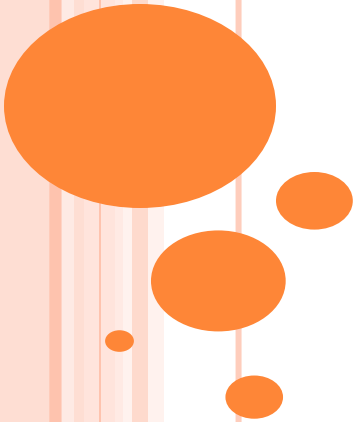
Antonio Martín Sierra

Único archivo de configuración

- **Desde Spring 3.5 se pueden definir todos los perfiles en un único .properties o .yml:**



Spring Cloud

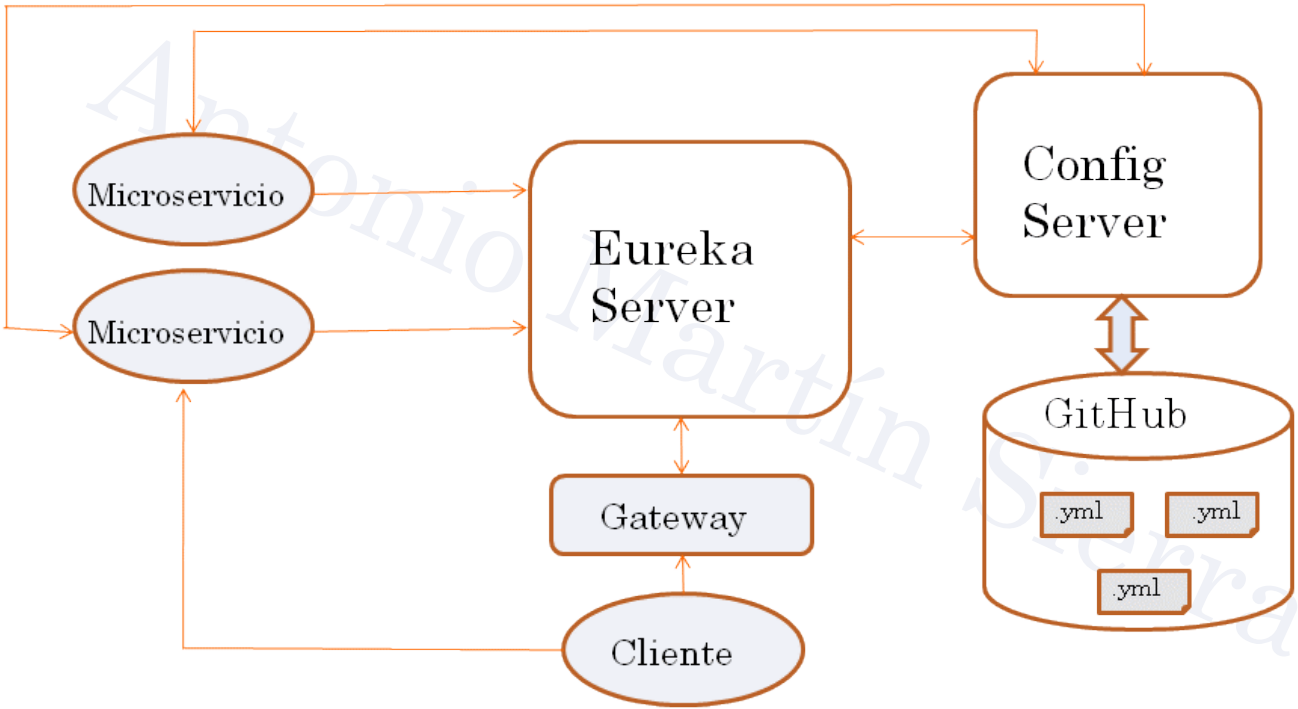


Tecnologías

- **Eureka Server. Servicio en el que se registran microservicios para facilitar el descubrimiento y utilización de los mismos**
- **Ribbon. Librería del lado cliente que proporciona balanceo de carga en el acceso a servicios**
- **Spring Cloud Config. Servidor que centraliza la configuración de un conjunto de servicios**
- **Gateway. Punto de acceso único a un conjunto de servicios**

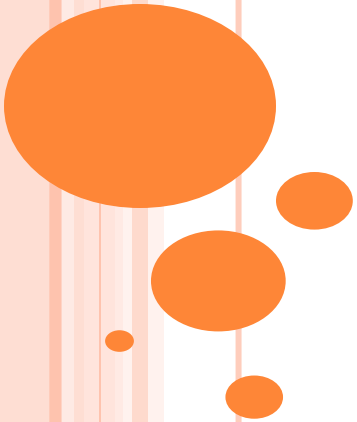
Antonio Martín Sierra

Esquema general



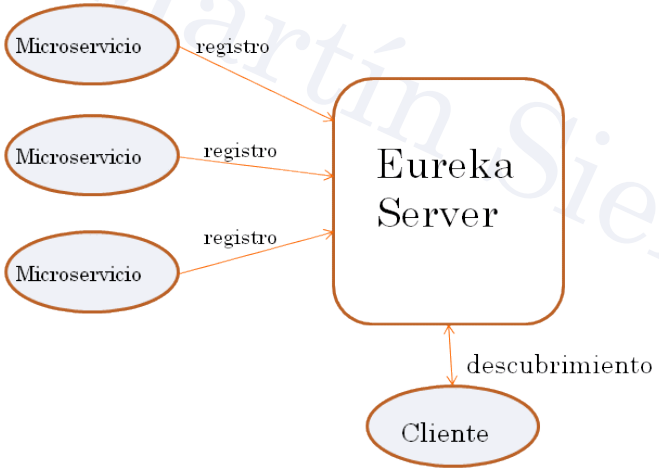
Antonio Martín Sierra

Servidor Eureka



Eureka server

- Servidor para registro y descubrimiento de servicios
- Permite acceder a los servicios sin conocer su ubicación real y puerto



Antonio Martín Sierra

Creación del servidor Eureka

- **Se crea como microservicio Spring Boot**
- **Requiere la dependencia Eureka server:**

```
<dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
```

- **Clase main anotada con** @EnableEurekaServer

```
@SpringBootApplication
@EnableEurekaServer
public class Application {
    public static void main(String[] args) {
            SpringApplication.run(Application.class, args);
    }
}
```

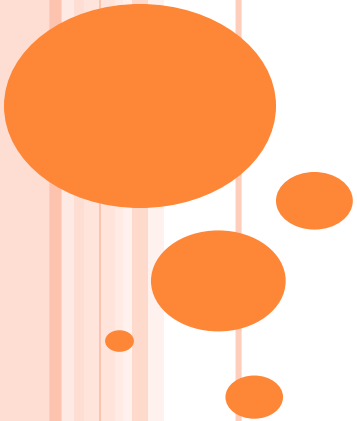
Antonio Martín Sánchez

Configuración del servidor Eureka

application.yml

```
spring:
  application:
    name: eureka-server
server:
  port: 8761
eureka:
  client:
    registerWithEureka: false
    fetch-registry: false
```

Registro y descubrimiento de servicios en Eureka



Registro de servicios en Eureka

- **Requiere la dependencia Eureka client:**

```
<dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

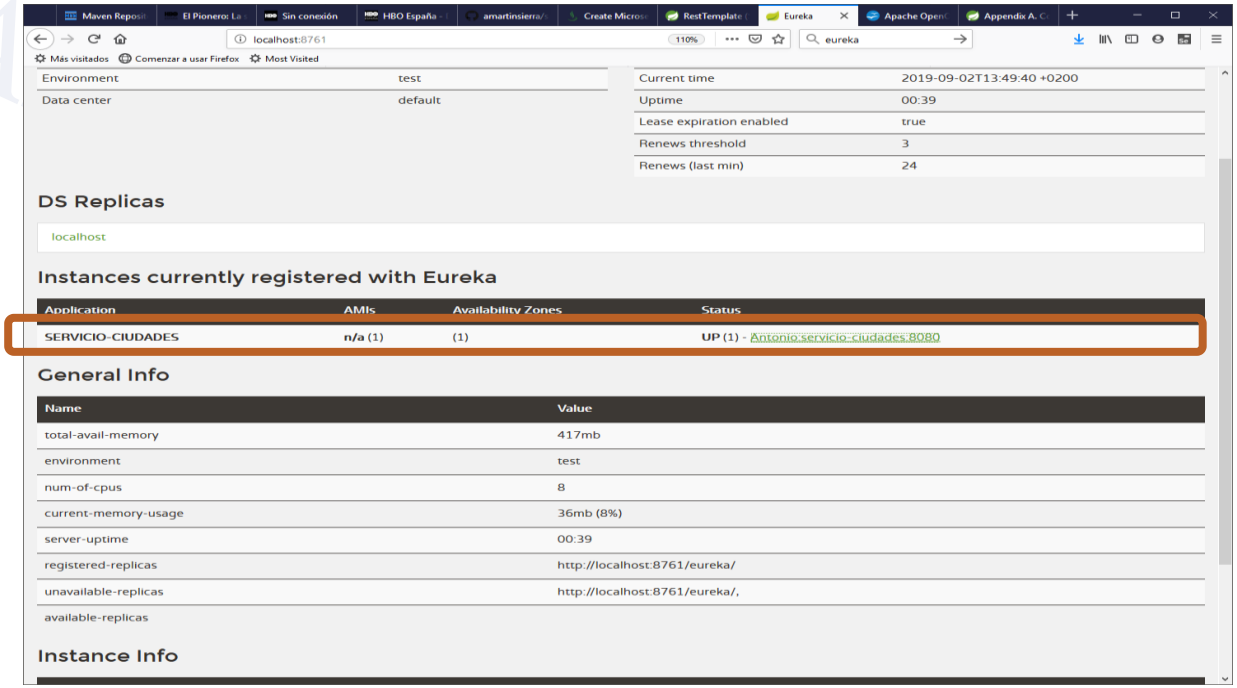
- **En el archivo de configuración properties o yml se indica el nombre de registro del servicio y la url del servidor Eureka:**

```
spring:
  application:
    name:  servicio-libros
server:
  port: 8080
eureka:
   client:
    serviceUrl:
      defaultZone: http://localhost:8761/eureka
```

Antonio Martín

Servicios registrados

http://localhost:8761



Descubrimiento de servicios

- **Al igual que el servicio registrado, el servicio cliente requiere la dependencia Eureka Client**

- **El método de creación RestClient.Builder anotado con @LoadBalanced para activar acceso por Eureka:**

Debe definirse un Builder adicional, que es el utilizado internamente por el propio Eureka

```
@Bean
@LoadBalanced
public RestClient.Builder loadBalanced(){
    return RestClient.builder();
}
@Bean
@Primary
public RestClient.Builder builder(){
    return RestClient.builder();
}
```

Descubrimiento de servicios

- **En la capa de servicio se obtiene RestClient a partir del Builder balanceado**

```
public EjemplaresServiceImpl(@LoadBalanced RestClient.Builder builder) {
        this.restClient =builder.build();
}
```

- **La url para acceder al servicio será el nombre del mismo, que es como se registra en eureka:**

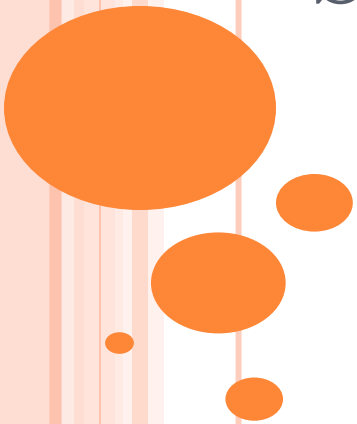
```
private String url="http://servicio-libros/";
```

- **En la configuración, el cliente puede elegir no registrarse en Eureka.**

```
:
eureka:
  client:
    register-with-eureka: false
    serviceUrl:
      defaultZone: http://localhost:8761/eureka
```

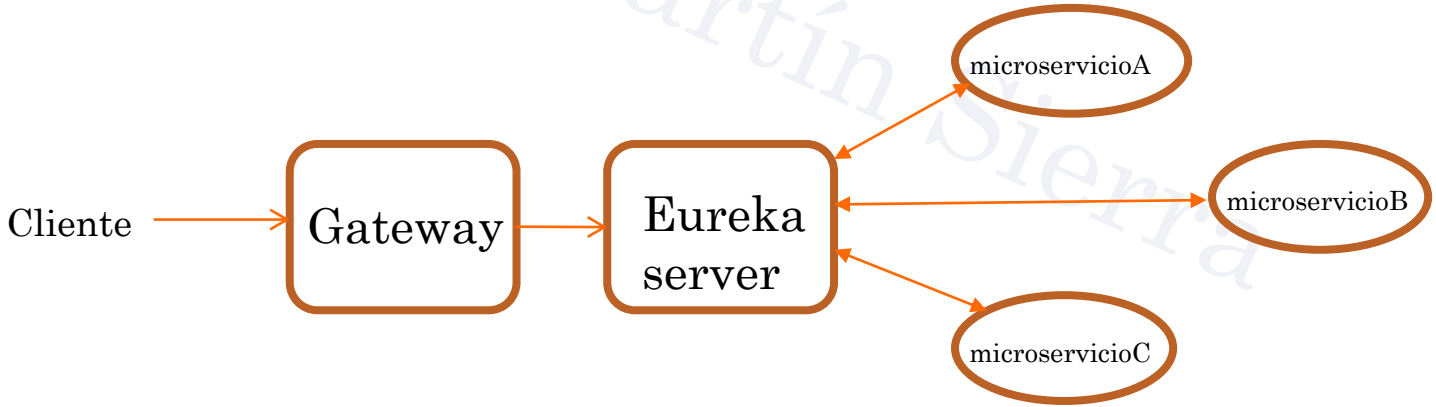
Jorge Martín Sierra

Spring Cloud Gateway



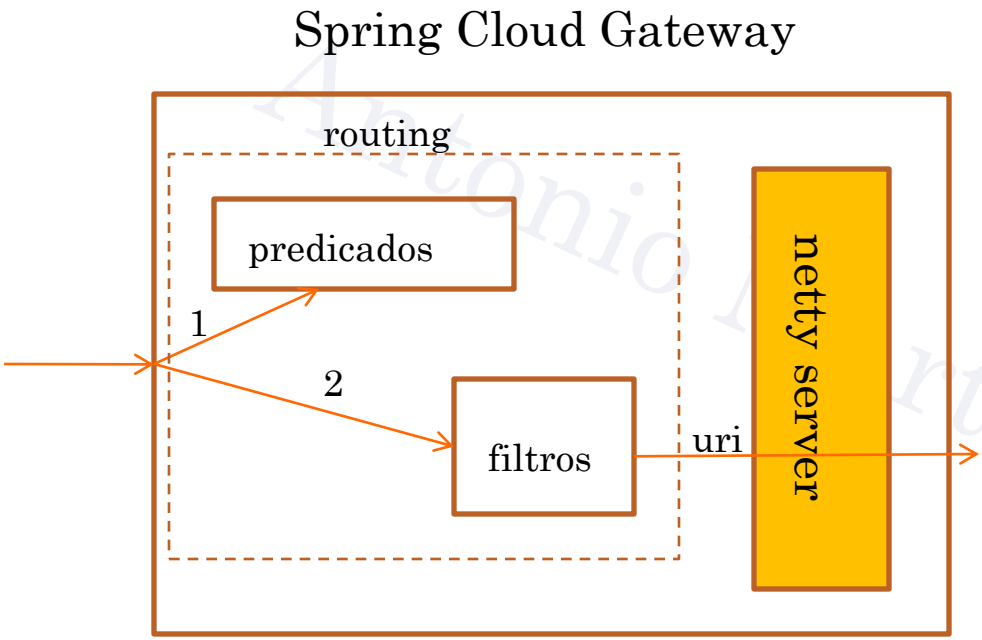
Características

- Servidor que proporciona un punto de acceso único a un conjunto de servicios.
- Utilizado desde Spring 2.4, antes Zuul Server.
- Acceso a los servicios a través de Eureka.



Antonio Martín Sierra

Arquitectura

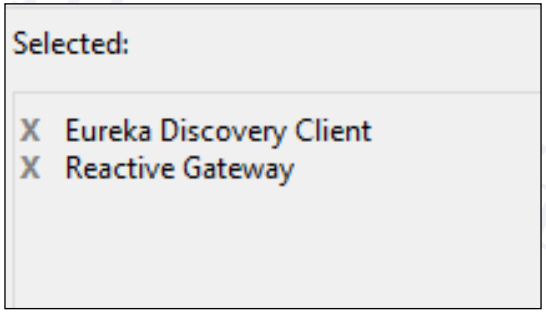


- A través de los *enrutados* (routing) se define lo que se debe hacer con una petición específica.
- El *predicado* establece la condición (path, cookies, encabezados,..), que debe cumplir una petición para ser enrutada a un determinado destino.
- Antes de dirigir la petición al destino, se le pueden aplicar uno o varios *filtros* que modifiquen alguna característica de la petición.

Antonio Martín Sierra

Creación del servidor Gateway

- Se crea como microservicio Spring Boot
- Requiere las dependencias Reactive Gateway y Discovery Client, no se debe incluir Spring Web:



- La clase main no tiene que tener ninguna anotación especial

Configuración

- **A través de spring.cloud.gateway.routes, se indican los mapeos de las peticiones a los servicios.**
- **Cada ruta incluye un identificador, dirección del servicio (uri) predicados y posibles filtros a aplicar:**

```
…
spring:
  cloud:
   gateway:
      routes:
      - id: serv-libros
          #lb:// cuando se va a acceder a través de Eureka
         uri: lb://servicio-libros#http://localhost:8080
         predicates:
          - Path=/slibros/**
         filters:
          - RewritePath=/slibros/*, /
…
```

Antonio M. Sierra

Configuración CORS

- **Para evitar bloque de peticiones desde script, se debe incluir una configuración especial:**

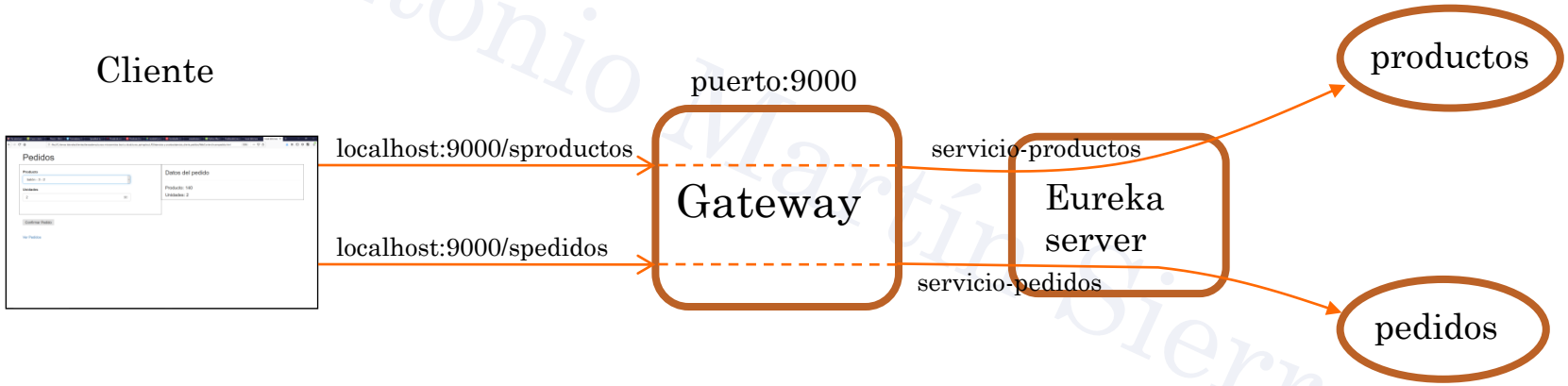
```
...
spring:
  cloud:
    gateway:
      default-filters:
        - DedupeResponseHeader=Access-Control-Allow-Credentials Access-Control-Allow-Origin
      globalcors:
        corsConfigurations:
          '[/**]':
            allowedOrigins: "*"
            allowedMethods:  "*"
            allowedHeaders:  "*"
      routes:
        ...



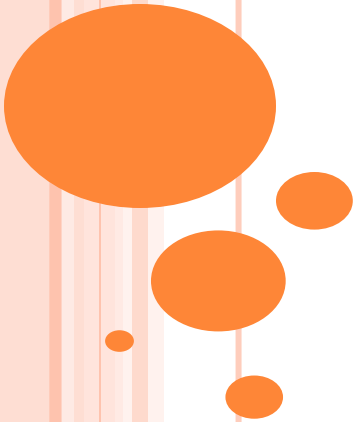
...
```

Utilización Gateway

Antonio Martín Sierra

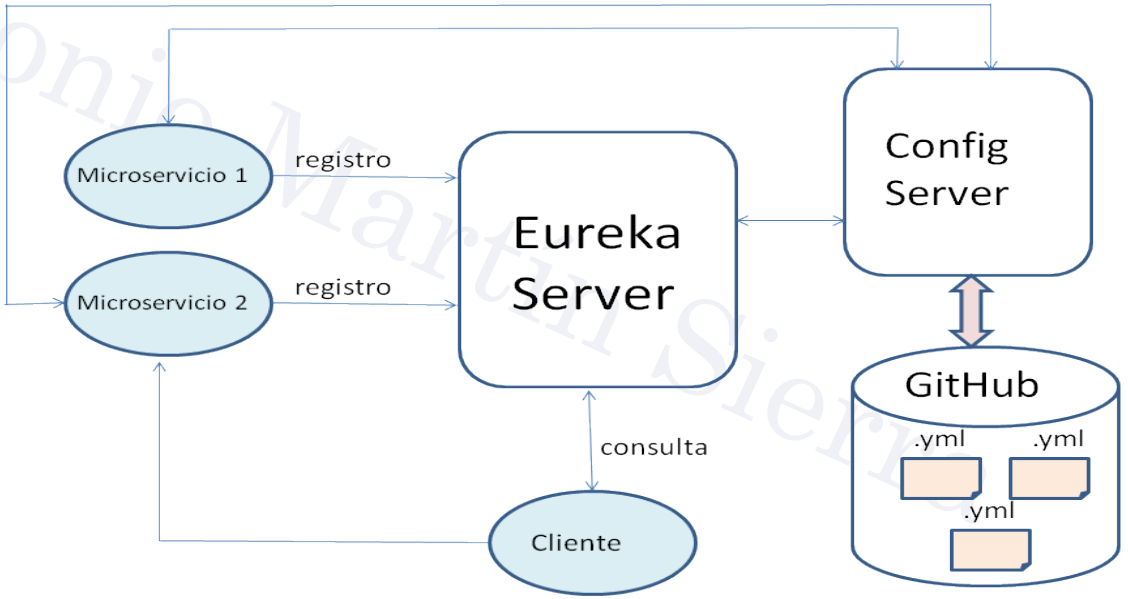


Servidor de configuración



Spring Cloud Config

- Centraliza la información de configuración de un conjunto de servicios dentro de un repositorio GitHub



Antonio Martín Sierra

Creación del servidor

- **Se crea como microservicio Spring Boot**
- **Requiere la dependencia Config Server:**

```
<dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-config-server</artifactId>
</dependency>
```

- **Clase main anotada con** @EnableConfigServer

```
@EnableConfigServer //lo habilita como servidor de configuración
@SpringBootApplication
public class Application {
        public static void main(String[] args) {
                SpringApplication.run(Application.class, args);
        }
}
```

Antonio Martín Sierra

Configuración del servidor

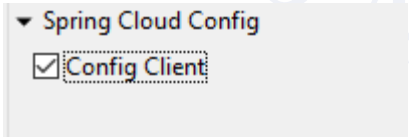
- **Mediante la propiedad spring.cloud.config se establece la localización del repositorio**

```
server:
  port: 8888
spring:
  cloud:
    config:
      server:
        git:
          uri: https://github.com/amartinsierra/spring-config
```

Configuración de los microservicios

- Los archivos de configuración de los servicios se llevan al repositorio de GitHub, nombrándolos como: nombre-servicio.yml

- Se incluirá el starter Config Client:

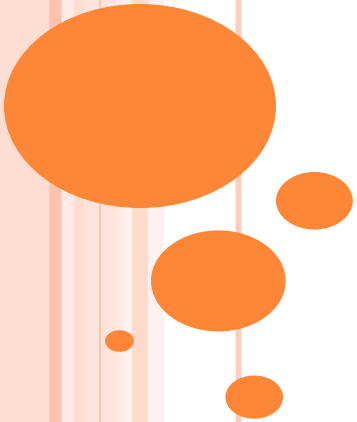


- En la configuración de los servicios:

```
spring:
  application:
    name: servicio-conf     # Nombre microservicio
  config:
    import: optional:configserver: http://localhost:8888  #Localización Spring Cloud Config
```

Antonio Martín

Despliegue de servicios REST en contenedores docker



¿Qué es un contenedor?

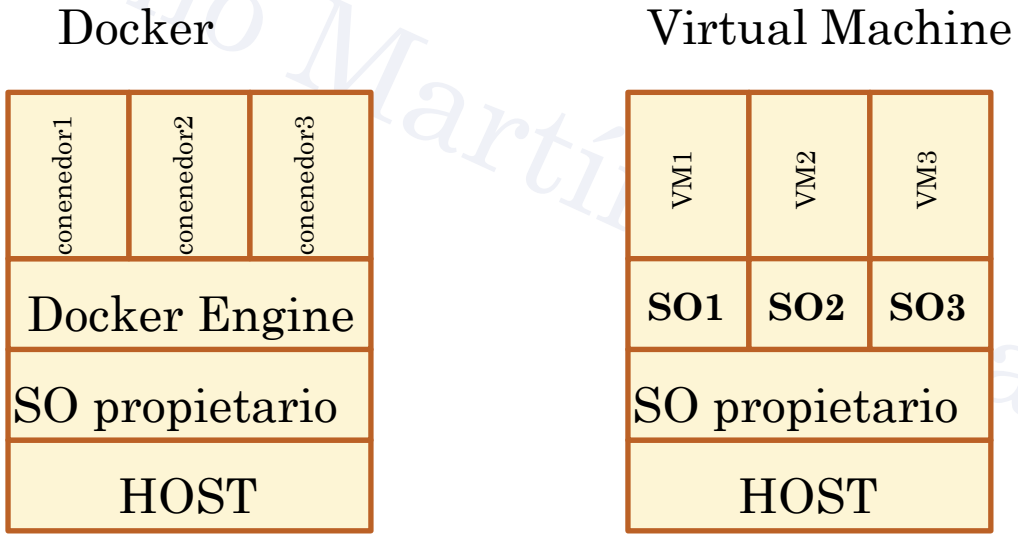
- Sistema que permite empaquetar una aplicación, junto con toda la infraestructura necesaria para su funcionamiento, facilitando su despliegue
- Los contenedores comparten el sistema operativo del host donde se ejecutan
- Más ligero, eficiente y portable que el sistema de máquinas virtuales

Antonio Martín Sierra

Contenedores docker

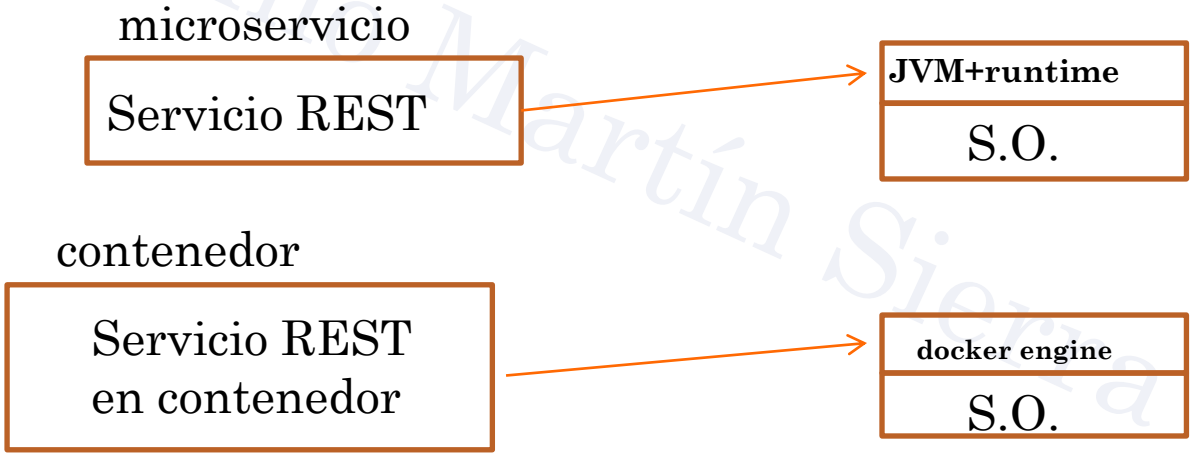
- **Docker es un entorno de ejecución para contenedores.**
- **Docker vs VM:**

Antonio Martín



Servicios REST en contenedores

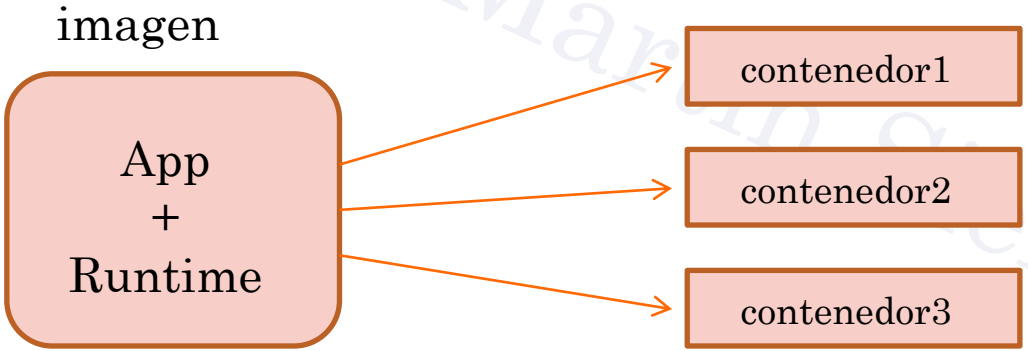
- **Al incluir un servicio REST en un contenedor, no tenemos que preocuparnos en instalar el intérprete de Java y las dependencias en la máquina.**



Antonio Martín Sierra

Imágenes y contenedores

Imagen	Contenedor
• Paquete formado por el programa y todo el software necesario para su funcionamiento	• Instancia de una imagen



Antonio Martín Sierra

Descarga e instalación

- **Podemos descargar docker desde la siguiente dirección:**

  https://www.docker.com/products/docker-desktop/

- **Una vez descargado docker desktop, se instala con las opciones por defecto.**

- **Tras instalarlo, lo iniciamos y comprobamos su funcionamiento mostrando la lista de imágenes:**

  >docker images

Antonio Martín Sierra

Comandos básicos

- **Docker puede manipularse desde línea de comandos mediante**

  >docker *comando*

- **Entre los principales comandos básicos están:**

>docker images	**Muestra una lista de todas las imágenes existentes en la máquina**
>docker ps	**Muestra los contenedores en ejecución. Si queremos mostrar también los detenidos: docker ps –a**
>docker run nombre_imagen	**Crea una nueva instancia de la imagen (contenedor) y la ejecuta. Si la imagen no existe en local, la descarga del docker hub**
>docker rmi imagen	**Elimina la imagen especificada. Si no se puede borrar porque hay contenedores en ejecución, se puede forzar el cierre con docker rmi –f imagen**
>docker rm contenedor	**Elimina un contendor parado. Si no está parado, primero hay que detenerlo con docker stop contenedor. Si se quieren borrar todos los contenedores:** docker rm $(docker ps -aq)

Antonio Martínez

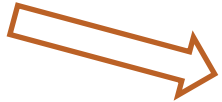
Creación de imágenes docker

El archivo dockerfile

- **Contiene la información necesaria para que docker pueda construir una imagen**
- **Se compone de una serie de comandos con sus correspondientes valores:**

estructura dockerfile

```
COMANDO1  valor
COMANDO2  valor
…
```



ejemplo dockerfile

```
FROM openjdk
ADD .  /app
CMD ["uvicorn","main:app",..]
```

Antonio Martín Sie

Principales comandos

- **FROM. Indica la imagen base de la que se parte**
- **COPY. Copia archivos y directorios:**

```
COPY ruta_origen  ruta_destino
```

⟹

```
COPY servicio.jar   apps/servicio.jar
```

- **ENTRYPOINT. Comando que debe ser ejecutado al lanzar el contenedor:**

```
ENTRYPOINT java -jar apps/servicio.jar
```

Antonio Martín Sierra

Ejemplo dockerfile

Se parte de una imagen con JDK instalado

```
FROM openjdk:21-ea-jdk-slim
COPY servicio.jar    apps/servicio.jar
ENTRYPOINT java -jar apps/servicio.jar
```

Martín Sierra

Construcción de la imagen

- **Para construir la imagen, utilizamos el comando *build* con el siguiente formato:**

```
>docker build -t nombre_imagen ruta_dockerfile
```

- **Si escribimos el comando en la misma carpeta donde está dockerfile:**

```
>docker build -t imagen1 .
```

- **Una vez construida, aparece en nuestro listado de imágenes con *docker images***

Antonio Martín Sierra

Creación y ejecución de un contenedor

- Para crea contenedores a partir de una imagen y ponerlos en ejecución utilizamos el comando *run*.

- En el caso de un servicio, deberá mapearse el puerto del servidor embebido a un puerto de la máquina física:

```
>docker run -p puerto_equipo:puerto_servidor nombre_imagen
```

- Por ejemplo, para crear un contenedor mapeado al puerto 8001 de la máquina física:

```
> docker run -p 8001:8000 imagen1
```

- Se le puede dar un nombre al contenedor:

```
> docker run -- name contenedor1 -p 8001:8000 imagen1
```

Antonio Martín Sierra

Definir variables de entorno

- En un microservicio, las variables de entorno se establecen en propiedades definidas dentro del archivo .properties o .yml
- Se asigna el valor ${nombre_variable} a la propiedad
- Por ejemplo:

```
spring.datasource.username=${USER_DB}
spring.datasource.password=${PASS_DB}
```

Antonio Martín Sierra

Asignar valores a variables

- **Cuando vamos a desplegar un microservicio a través de contenedores docker, los valores de las variables de entorno se pueden asignar:**

  - **Durante la construcción de la imagen en dockerfile:**

```
ENV USER_DB  root
ENV PASS_DB  root
```

  - **Si en el momento de crear la imagen los valores de las propiedades aún no se conocen, se pueden asignar durante la creación del contenedor:**

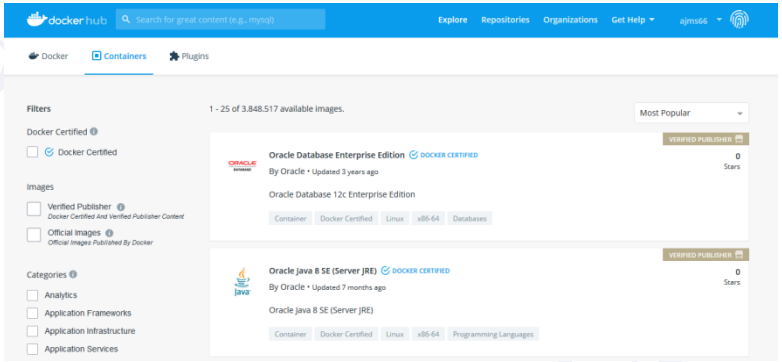
```
>docker run -e USER_DB=root -e PASS_DB=root -p 9000:8080 imagenservice
```

Antonio Martín Sierra

Docker hub

Fundamentos

- Repositorio público en la nube que cuenta con miles de imágenes disponibles para libre uso

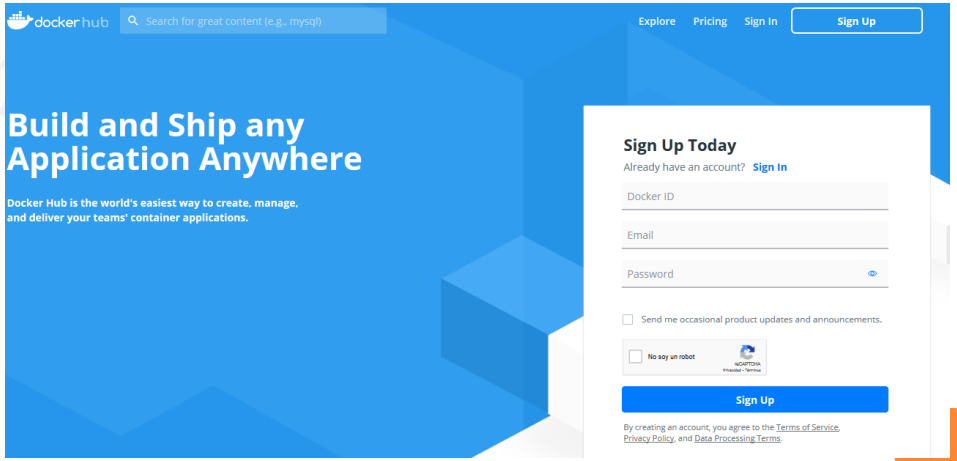


- Es posible subir nuestras imágenes a docker hub para poder desplegarlas en cualquier equipo

Creación de una cuenta

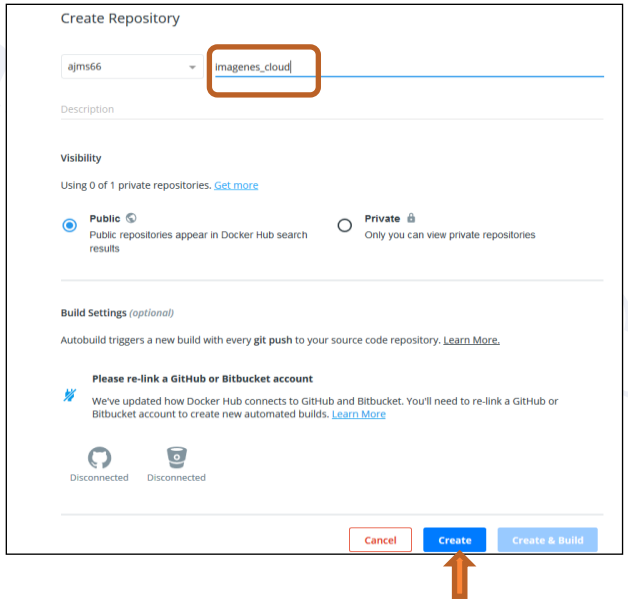
- **Para poder subir imágenes a docker hub, debemos crear primero una cuenta gratuita:**

https://hub.docker.com/



Creación de un repositorio

- **Para subir imágenes a docker hub, debemos crear un repositorio en el que serán guardadas**



Subida de imágenes a docker hub

- **Se deberán seguir los siguientes pasos:**
  - **Autenticación en docker hub**

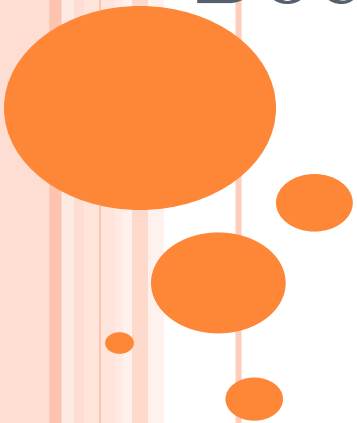
    >docker login -u *usuario*
  - **Etiquetar la imagen**

    >docker tag imagen *usuario/repositorio:etiqueta*
  - **Subir la imagen**

    >docker push *usuario/repositorio:etiqueta*

Antonio Martín Sierra

Dockerizar clientes eureka

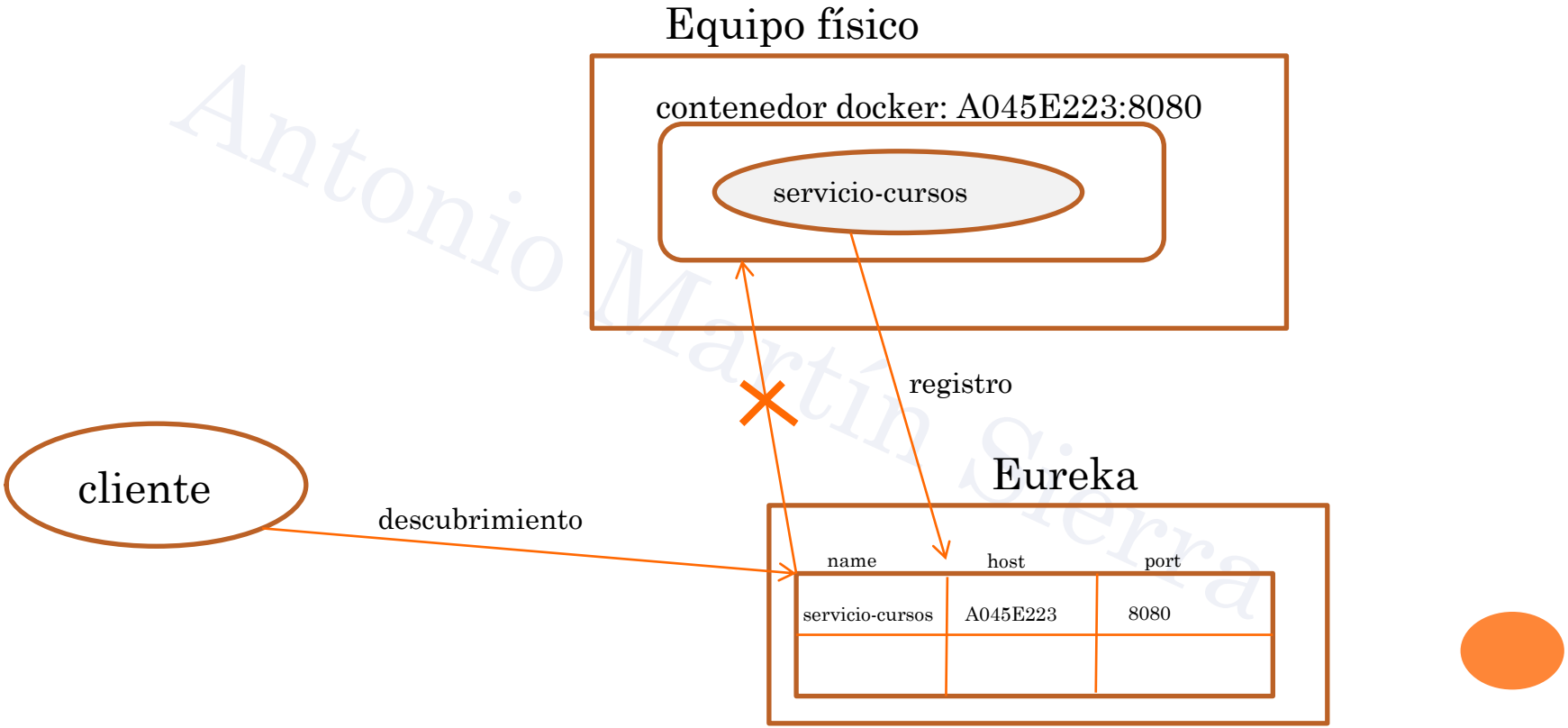


Problemática

- Cuando un microservicio se registra en eureka, lo hace con el nombre de host y puerto del entorno de ejecución.
- Si el microservicio se está ejecutando en un contenedor docker, se registrará con el nombre y puerto del contenedor.
- Problema: Cuando el cliente descubra su localización en eureka intentará acceder al contenedor, y este no es accesible externamente.

Antonio Martín Sierra

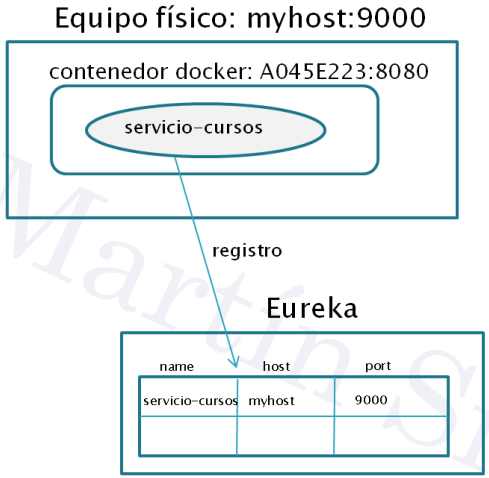
Grafica del problema



Antonio Martín Sierra

Solución

- **El microservicio debe registrarse con el nombre y puerto del host o equipo físico:**

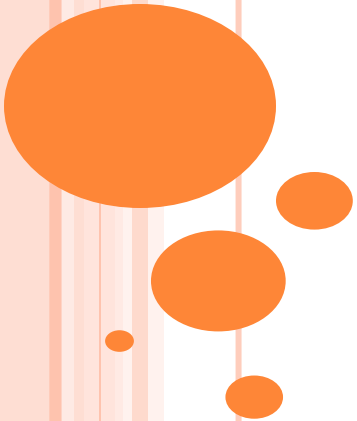


- **Se deben configurar las siguientes propiedades:**

```
eureka.instance.hostname=myhost    #recomendable usar variables de entorno
eureka.instance.non-secure-port=9000   #recomendable usar variables de entorno
```

Antonio Martín Sierra

Observabilidad en microservicios Spring Boot



Observabilidad de aplicaciones

- Seguimiento y recopilación de datos de una aplicación en tiempo real.
- Ayuda a detectar fallos y cuellos de botella en aplicaciones Web.
- Mediante el uso de herramientas de observabilidad, se recopilan datos de un programa en ejecución que pueden ser monitorizados y analizados a fin de detectar posible problemas.

Antonio Martínez Sierra

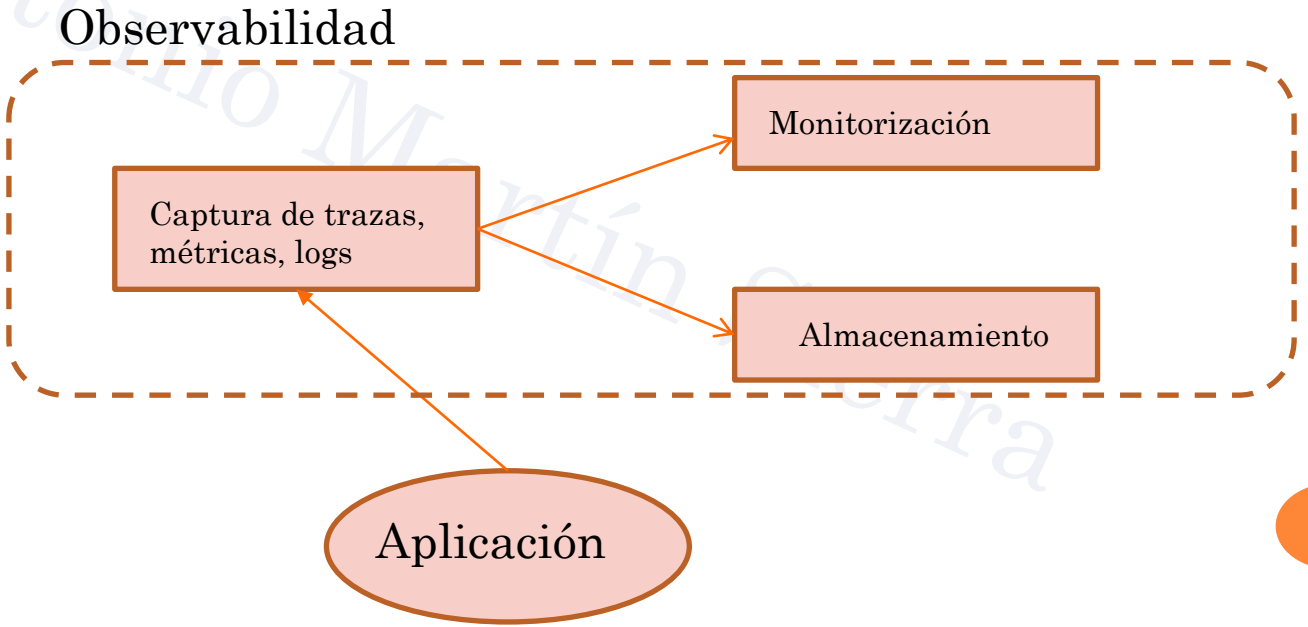
Datos observados

- **En un sistema de observabilidad se distinguen tres tipos de datos:**
  - **Métricas. Medidas que se toman en el sistema: Número de peticiones recibidas, tiempo de respuesta, etc.**
  - **Trazas. Permiten realizar un seguimiento de por donde va pasando la petición, tiempo que tarda en realizar una acción, etc.**
  - **Logs. Eventos que se van registrando en la aplicación durante su ejecución (errores, información, etc.)**

Antonio Martín Sierra

Sistema de observabilidad

- **Un sistema de observabilidad incluye herramientas para captura de trazas y métricas, así como para su almacenamiento y monitorización**



Antonio Martín Ocaña

Librerías y herramientas

- **En el contexto de aplicaciones Spring Boot, existen numerosas librerías y herramientas para realizar una trazabilidad de aplicaciones:**
  - **Actuators. Librería que proporciona endpoints para obtener información sobre el estado del sistema**
  - **Micrometer. Librería en captura de métricas para ser analizadas por otros sistemas de monitoreo**
  - **Prometheus. Herramienta par almacenamiento de métricas**
  - **OpenTelemetry. Estándar para captura de trazas**
  - **Grafana. Herramienta para visualización de métricas**
  - **Jaeger. Herramienta para visualización de trazas**

Antonio Martín Sierra

Actuators

- **Nos permite acceder a distinta información sobre el sistema.**
- **Se requiere la dependencia:**

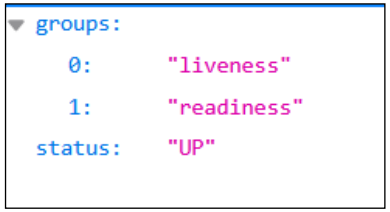
```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

- **En el archivo properties incluimos la siguiente propiedad para capturar distintos elementos:**

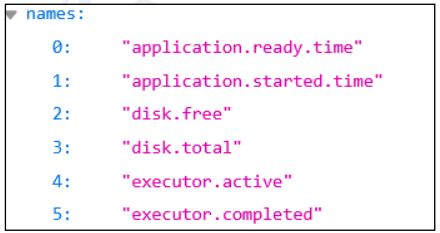
```
management.endpoints.web.exposure.include=health,info,metrics,prometheus
```

Actuators - Enpoints

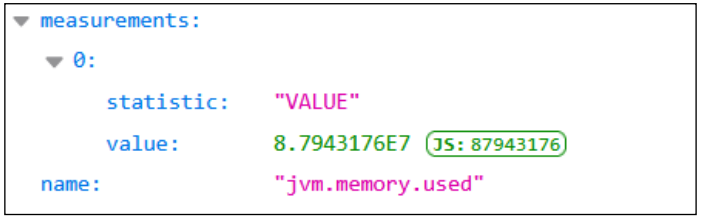
- http://localhost:8080/actuator/health



- http://localhost:8080/actuator/metrics



- http://localhost:8080/actuator/metrics/jvm.memory.used



Antonio Sierra

Micrometer

- **Permite definir métricas personalizadas en la aplicación.**
- **Se requiere la dependencia:**

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-micrometer-metrics</artifactId>
</dependency>
```

- **Proporciona los siguientes objetos para captura de métricas personalizadas:**
  - **Counter: Utilizado para contabilizar las peticiones a un determinado recurso**
  - **Timer: Permite conocer la duración del procesamiento de un determinado bloque de código**

Ejemplo uso micrometer

```
@RestController
public class ProductoController {
    private final Counter consultas;
    private final Timer tiempoListado;
    public ProductoController(MeterRegistry registry) {
        this.consultas = Counter.builder("productos.consultas")
                .description("Número de consultas al catálogo")
                .register(registry);
        this.tiempoListado = Timer.builder("productos.listado")
                .description("Duración de la consulta del catálogo")
                .register(registry);
    }
    @GetMapping
    public List<String> listar() {
        return tiempoListado.record(() -> {
            consultas.increment();
            return List.of("Teclado", "Ratón", "Monitor");
        });
    }
    :
```

nombre de las métricas

registro tiempo de ejecución

incremento contador

http://localhost:8080/actuator/metrics/productos.consultas

http://localhost:8080/actuator/metrics/productos.listado

Prometeus

- Recoge métricas y las almacena para procesamiento.
- La siguiente dependencia permite publicar métricas:

```
<dependency>
        <groupId>io.micrometer</groupId>
        <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

- Podemos observar el listado de métricas en formato prometheus en:

localhost:8080/actuator/prometheus

A. Martín Sierra

Almacenamiento en Prometeus

➢Se define el archivo prometheus.yml

```
global:
  scrape_interval: 5s
scrape_configs:
  - job_name: "productos-api"
    metrics_path: /actuator/prometheus
    static_configs:
      - targets: ["host.docker.internal:8080"]
```

Prometheus consulta periódicamente esta dirección para obtener métricas

➢Creamos el contendor:

```
docker run -d --name prometheus -p 9090:9090
 --mount "type=bind,source=%cd%\prometheus.yml,
                target=/etc/prometheus/prometheus.yml,readonly" prom/prometheus
```

➢En http://localhost:9090 se consultan las métricas

- **http_server_requests_seconds_count. Peticiones atendidas**
- **productos_listado_seconds_count. Peticiones recurso**

Grafana

- Recoge métricas de prometheus y las muestra en gráficas
- Se instala mediante contenedor docker:

```
docker run -d --name grafana -p 3000:3000 grafana/grafana
```

- En localhost:3000 accedemos a la consola de administración de grafana utilizando las credenciales por defecto admin/admin

Antonio Martín Sierra

Grafana. Añadir gráficos

- **Primero añadimos fuente de datos:**

  Connections → Data sources y selecciona Add data source → Prometheus.

- **En url introducimos**

  http://host.docker.internal:9090

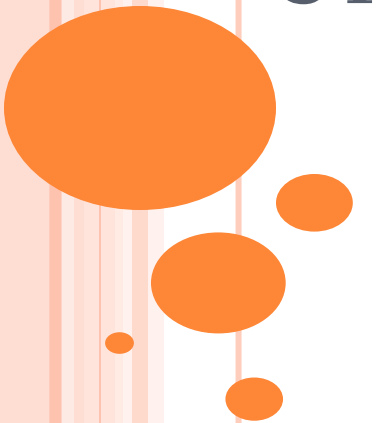
- **Creamos un dahsboard:**

  Dashboards → New → New dashboard → Add visualization

- **Añadimos prometheus como fuente de datos y en metrics elegimos la métrica a visualizar**

Antonio Martín Sierra

Utilizar OpenTelemetry

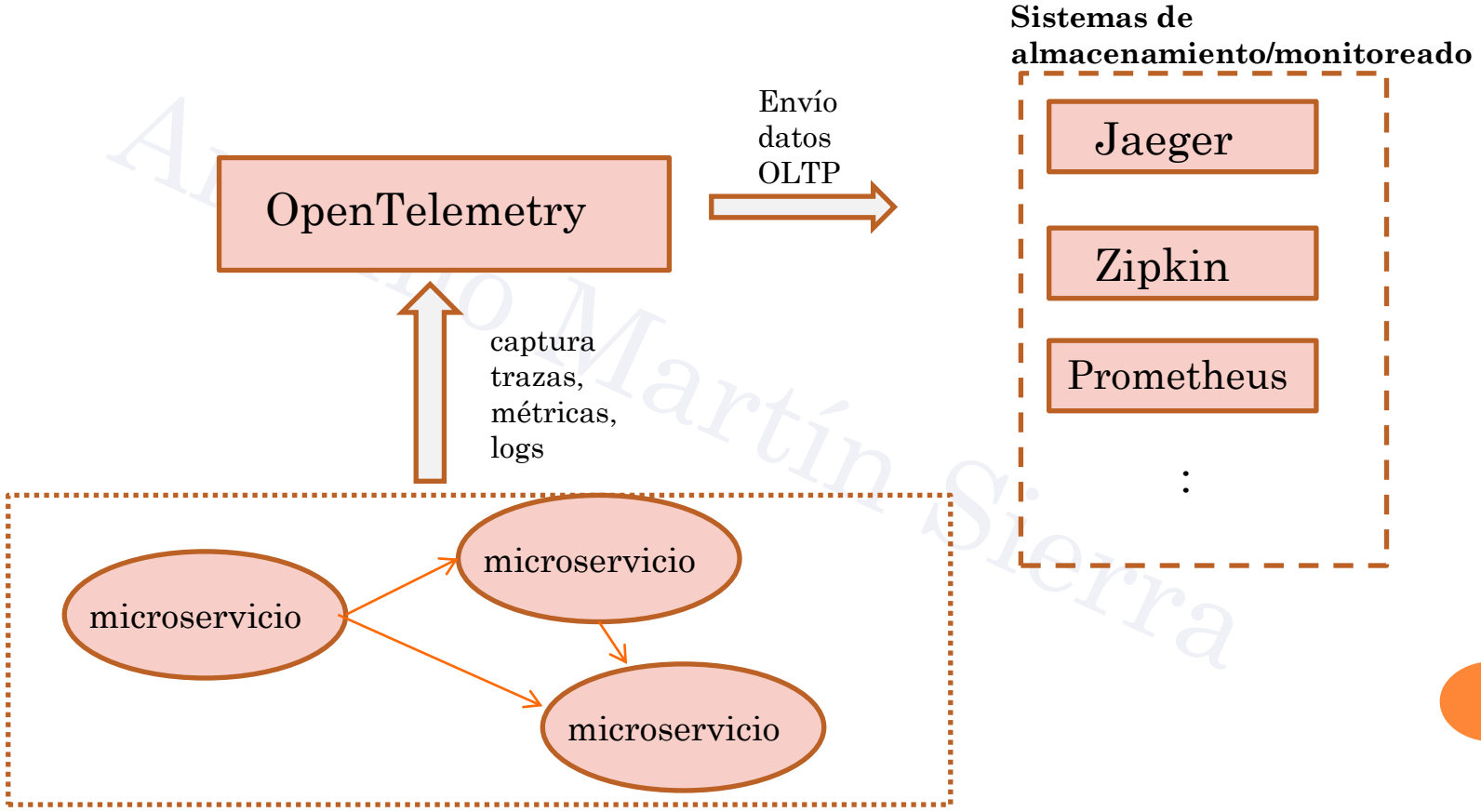


OpenTelemetry

- Estándar unificado para la captura de trazas y métricas en aplicaciones distribuidas.
- Utiliza un protocolo independiente de plataforma (OTLP), que permite enviar datos (trazas, métricas y logs) a diferentes backends (Jaeger, Zipkin, Prometheus,etc.).
- Integrado en Spring Boot.
- Modo de funcionamiento automático y manual.

Antonio Martín Sierra

Esquema OpenTelemetry



Antonio Martín Sierra

Dependencias

- **Se deben incluir las siguientes dependencias para utilizar OpenTelemetry en aplicación Spring Boot:**

BOM

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>io.opentelemetry.instrumentation</groupId>
      <artifactId>opentelemetry-instrumentation-bom</artifactId>
      <version>2.6.0</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

starter

```
<dependency>
   <groupId>io.opentelemetry.instrumentation</groupId>
   <artifactId>opentelemetry-spring-boot-starter</artifactId>
</dependency>
```

Instrumentar la aplicación

- **OpenTelemetry instrumenta automáticamente (incluye trazas de forma implícita) varios elementos de la aplicación: controladores HTTP, métodos de repositorio y objetos RestClient, WebClient y RestTemplate.**

- **Se pueden incluir span o trazas manuales mediante la anotación io.opentelemetry.instrumentation.annotations.WithSpan:**

```
@WithSpan(value = "metodo_pedidos")
@Override
public List<Pedido> pedidos() {
    return pedidosDao.findAll();
}
```

Antonio Martín Sierra

Configuración

- El starter OpenTelemetry incluye configuraciones por defecto.
- De forma predeterminada, los datos son enviados a la dirección http://localhost:4318
- Para modificar esta propiedad:

```
otel.exporter.otlp.endpoint=http://ip:puerto
```

- Por defecto se envía todo, trazas, logs y métricas. Si se quiere desactivar logs:

```
otel.logs.exporter=none
```

Antonio Martín Sierra

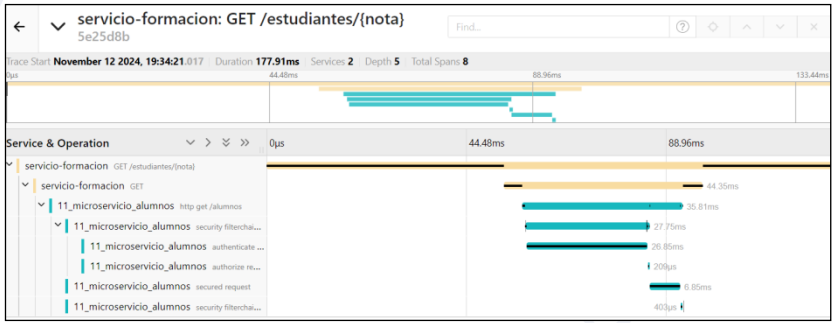
Monitorieado de datos

- Se puede utilizar cualquier servidor de monitorieado compatible con OTLP, por ejemplo Jaeger, para visualizar los datos enviados.

- Para ejecutar Jaeger mediante contenedor docker:

```
docker run --rm -d --name jaeger
-p 16686:16686
-p 4317:4317
-p 4318:4318
jaegertracing/all-in-one:1.53
```

http://localhost:16686/



- Jaeger solo monitoriza trazas